

Seasons in Teamder — *continued*

9. Season scope versus lifetime — how the statistics are divided

Every number on a club screen comes from one of four places: a **live scan** over the club's finished game documents (`gameService.getCommunityStats`), a **live rollup** of per-player counters (`communityPlayerStats` , read through `gameService.getCommunityChampionship`), a **sealed archive** of one closed season (`seasonSummary/{groupId}__{seasonId}` , plus its small twin `seasonCards`), or a **merge** of the live rollup with every archive (`src/utils/allTimeTable.ts`). Closing a season zeroes the rollups and leaves the scan untouched, so the divide between "this season" and "all time" is not one mechanism — it is two different mechanisms that have to be kept in agreement by hand, on every screen, for every row. This section is about the scan, which is the half that was retro-fitted with a season filter, and about the five places that call it.

Where a defect was fixed after the shipped store binaries were cut, that is stated: both 1.1.9 artefacts were built from `4de80d0` , which predates most of what follows (ROUND4 #1, and the front matter's *What is running where*).

`gameService.getCommunityStats` — one query, two tallies

`src/services/gameService.ts:949–1262` . The signature is one required club id and one optional season scope:

```
async getCommunityStats(
  groupId: GroupId,
  season?: { currentId: string; currentNo: number }, // :965
): Promise<{ /* ~14 fields, listed below */ }>
```

The query (`:1112–1117`) is fixed and bounded:

```
const q = query(
  col.games(),
  where('groupId', '==', groupId),
  where('status', 'in', ['finished', 'cancelled']),
  orderBy('startsAt', 'desc'),
  limit(200),
);
```

So the universe is **the 200 most recent terminal game documents of this club, newest first**. Nothing paginates past that. A permission-denied read returns the all-zero `empty` object (`:1008–1033`); any other error is logged to the `errors` inbox and re-thrown, which every caller turns into `null` with a `.catch` .

The loop (`:1150–1217`) runs **once** and maintains **two independent tallies**. The lifetime tally is computed first, in a bare block *before* the season gate:

```

for (const doc of snap.docs) {
  const g = doc.data();
  {
    // :1155 – lifetime, unscoped
    const st = eveningPlayState(g as PlayableEvening);
    if (g.status === 'cancelled' || st === 'notHappened') lifeCancelled += 1;
    else if (st !== 'unverified') {
      lifeFinished += 1;
      /* attendance set, 30d / 365d windows, lifeNights.push(...) */
    }
  }
  if (!inSeason(g as { seasonId?: string }, season)) continue; // :1171 – the gate
  const state = eveningPlayState(g as PlayableEvening); // :1179
  if (g.status === 'cancelled') { totalCancelled += 1; continue; }
  if (state === 'unverified') continue; // :1188 – counts for neither
  if (state === 'notHappened') { totalCancelled += 1; continue; } // :1191
  totalFinished += 1; // :1195
  /* thisMonthFinished, attendedTally, activeMonth, activeYear, nights.push(...) */
}

```

Two things are worth naming here. First, *whether an evening happened* is never re-derived: both tallies ask `eveningPlayState (src/utils/eveningPlayed.ts)`, which returns `'happened'`, `'notHappened'`, `'unverified'` or `'pending'`. An `'unverified'` night — auto-closed by the sweep with no timer, no goals and no rotation, and no admin ruling — is excluded from **both** halves of the organisation-rate fraction rather than counted as a failure. Second, the lifetime block has no `thisMonthFinished`, no `attendedByUser`, no `avgAttendance` and no `topPlayers`: those four exist only in the season-scoped half, which is why a caller that wants an unscoped attendance count must pass no season at all rather than read `lifetime`.

The return (:1230–1260) is:

Field	Scope	Meaning	Absent →
<code>totalFinished</code>	season	evenings <code>'happened'</code> in scope	0
<code>totalCancelled</code>	season	<code>cancelled</code> + <code>notHappened</code> in scope	0
<code>organizationRate</code>	season	<code>finished / (finished + cancelled)</code> , 0 when both are 0	0
<code>avgAttendance</code>	season	member arrivals ÷ finished nights	0
<code>thisMonthFinished</code>	season	finished within 30 days	0
<code>activeThisMonth</code>	season	distinct attendees, 30-day window $\cap$ season	0
<code>activeThisYear</code>	season	distinct attendees, 365-day window $\cap$ season	0
<code>topPlayers</code>	season	top 5 {uid, attended}, desc	[]

Field	Scope	Meaning	Absent →
<code>attendedByUser</code>	season	uid → nights attended (the authoritative "הופעות")	<code>{}</code>
<code>longestStreak</code> / <code>longestStreakUid</code>	season	longest consecutive-night run	0 / <code>null</code>
<code>currentStreakByUser</code>	season	uid → current run	<code>{}</code>
<code>lifetime.totalFinished</code> / <code>.totalCancelled</code> / <code>.organizationRate</code>	lifetime	as above, unfiltered	0
<code>lifetime.activeThisMonth</code> / <code>.activeThisYear</code>	lifetime	as above, unfiltered	0
<code>lifetime.longestStreak</code> / <code>.longestStreakUid</code>	lifetime	the club's permanent record	0 / <code>null</code>

The streak is computed by one shared helper, `lifetimeStreak` (:550–575), called twice — once on `nights` (:1218) and once on `lifeNights` (:1239) — so the season record and the lifetime record can never be computed by two different rules. It walks nights oldest→newest, resets a player's run on a missed night, and keeps the first uid to reach each new maximum (`if (run[uid] > longestStreak)` is strict). On a club where several regulars never miss, the named holder is therefore decided by `g.players` array order in the earliest night of the window — ROUND4 #28.

`limit-200-window-is-a-lifetime-label`. The doc-comment at :934–947 is honest that this is a window, not all time, but the field is named `lifetime` and the screen renders it under the word "אי פעם" ("ever"). For a club past 200 terminal documents — roughly four years of weekly play, or one year for a club that also cancels — `lifetime.totalFinished` silently becomes "the last 200 attempts". Worse under scoping: the `limit(200)` is applied by Firestore **before** `inSeason` runs in JavaScript, so a season that sits entirely outside the newest 200 documents scores zero with no error. For the one production club (23 terminal docs) neither bound bites.

`avgAttendance` **counts members only** (ROUND4 #16). The inner loop iterates `g.players` (:1207); guests live in a separate `g.guests` array whose ids never appear there. I read all 23 of the club's game documents and counted both arrays:

evening (UTC)	game	players	guests
2026-06-28 08:00	<code>WhqMQzLznMgP</code>	7	1
2026-06-29 20:30	<code>5EQKVHge5mtP</code>	2	6
2026-07-06 16:00	<code>2SMrLCGHyjnC</code>	6	7
2026-07-14 08:45	<code>fnBXPo76y0lm</code>	7	5
2026-07-16 17:00	<code>MMtE8J6HCLLH</code>	2	1
2026-07-27 07:20	<code>weK0fe0dMpbw</code>	7	5
2026-07-28 08:00	<code>79QqBt2YN0eg</code>	7	5

evening (UTC)	game	players	guests
2026-08-03 17:00	qVfvNY9mJijZ	7	11
2026-08-04 11:15	FCa5UtSdIb7m	7	1
2026-08-10 11:05	P7caI9Z8T4Qo	7	1
2026-08-23 06:45	Kea1X9HYgDS6	7	9
2026-08-24 12:10	VK9w206yPgpD	7	9
2026-08-26 10:20	T7qRTLr03I19	7	10
2026-08-31 13:35	9l51Te5dFqxL	7	5
2026-09-07 05:25	6HMqED17G8zj	7	2
2026-09-08 06:35	Wy5yCNMsxeYq	7	5
2026-09-11 05:35	ZMj4mA7rXZEJ	7	2
2026-09-14 09:40	hdirRxLgEXhj	7	6
2026-09-15 13:00	kcgHMaHeVLtX	7	6
2026-09-16 06:20	yA48XsyCEPCj	6	0
2026-09-17 14:30	nRHEgxVt3jYJ	7	1
2026-09-17 14:35	TZ7IWEJWvw2	6	0
(not held) 2026-09-17 21:19	DTNscolRojYD	1	1
22 held evenings		141	98

The loop skips a player whose arrival is `no_show`, and none of these evenings has one: the scan's own per-player tally, $22+20+20+20+20+20+19$, sums to the same **141**, so the raw slot count and the counted count agree to the unit. The screen therefore renders $141/22 = 6.4$ for nights that averaged $(141+98)/22 = 10.9$ on the pitch, and one of them — 03.08 — was 7 members and 11 guests. The label is "ממוצע הגעות למחזור" ("average arrivals per game-night"), and the club's own sealed archive records `guestGoals: 4`, so guests are participants elsewhere.

The season predicate

`src/utls/seasonScope.ts` holds the whole rule:

```
export function inSeason(game: { seasonId?: string }, season?: SeasonScope): boolean {
  if (!season) return true;
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === season.currentId : season.currentNo === 1;
}
```

Three cases. **No scope** → everything passes, and the function degenerates into the pre-seasons lifetime scan. A **stamp** → exact string equality with the running season id. **No stamp** → the evening belongs to season 1, because the `seasonId` stamp is written by the server only from the feature's ship date onward, and "your history becomes season 1" is what enabling seasons promises.

`unstamped-season-1-rule-disagrees-with-the-server-by-19` — **now fixed, and the fix has a sharp edge**. The server's copy is `eveningInSeason` in `functions/src/seasonCounters.ts:53–60`, character for character the same rule, and `functions/src/index.ts:15599–15637` (`playedEveningsOfSeason`) now passes `seasonNo` through to it. Before that fix the server counted only stamped games and answered **3** where the client answered **22**; a reopen of season 1 would have written 3.
`tests/logic/seasonCounterReconciliation.test.ts` runs both copies over one fixture.

The edge is what the rule does for a club **past** season 1. Production, `HhzIwmjMl1i5HSOGht3p` (מועדון שכחת שושי), read live:

```
groups/HhzIwmjMl1i5HSOGht3p.seasons =
  { enabled: true, currentId: 's2', currentNo: 2, count: 1,
    playedRounds: 0, roundsAtStart: 10, startedAt: 1789679141952,
    cadence: { type: 'rounds', targetRounds: 24 } }
```

23 terminal game documents. Their stamps: **19 carry none, 3 carry 's1', 1 carries 's3'** (an orphan — the club's seasons block has never known an `s3`). With `season = {currentId:'s2', currentNo:2}`, `inSeason` returns false for all 23: the unstamped ones fail `currentNo === 1`, the `s1` and `s3` ones fail the string compare. **Not one game in the club's history is in scope for the season it is currently playing**. Every season-scoped field above is therefore `0 / {} / null`.

A structural consequence nobody has written down: the stamp is written by `onGameRosterChanged` only on a transition into `active` or `finished` (`functions/src/index.ts:5802–5810`). A game cancelled from `open` never reaches either, so it is never stamped, so for any club in `season ≥ 2` a genuine cancellation can never enter `totalCancelled`. The season-scoped `organizationRate` is therefore **either exactly 0 (no evenings in scope) or exactly 1.0** — the only way to lose points is the `notHappened` route, an evening that kicked off and was later declared not to have happened. It is not a rate; it is a two-valued flag wearing a percentage sign.

The five callers

Caller	Passes a season?	Scope it gets	What it renders
<code>CommunityStatsScreen.tsx:199</code>	yes (:193–197)	season, with <code>lifetime</code> beside it	hero evening tile, <code>mostLoyal</code> , org donut, streak row, "פעילים השנה", <code>attendedByUser</code> → league table, badges, club level
<code>playerCompareService.ts:159</code>	yes (:146–152)	season	the <code>games</code> / "מחזורים" row of the compare card, and the denominator of every per-game rate on it
<code>CommunityDetailsScreen.tsx:189</code>	no	lifetime	"מפגשים שנערכו" headline, and the four-cell <code>CommunityStatsSection</code>

Caller	Passes a season?	Scope it gets	What it renders
assistantInsightsService.ts:170	no	lifetime	the coach's attendance streak and "ערבים על הדשא N" lines
SeasonsSettings.tsx:341	no	lifetime	history , the club's pre-seasons evening count shown in the activation confirm sheet

Two of those five are deliberate and correct, and three are deliberate and unscoped. The two season-passing callers both read the club document first for the sole purpose of learning the scope, and both carry a comment saying so — one extra document read per screen.

CommunityStatsScreen (:182–200) resolves the scope from the group:

```
const g = await groupService.get(groupId).catch(() => null);
const scope = g?.seasons?.enabled && g.seasons.currentId
  ? { currentId: g.seasons.currentId, currentNo: g.seasons.currentNo ?? 1 }
  : undefined;
```

(The local `scope` shadows the screen's `scope` state for the length of the effect; harmless, but it is the same word meaning two different things eight lines apart.) A club with seasons switched off passes `undefined` and gets its lifetime numbers in both halves, which is why the same function serves both kinds of club.

CommunityDetailsScreen is the screen the owner's original complaint was about. It calls with no season (:189) and renders `communityStats.totalFinished` as `matchesHeld` (:614) plus a four-cell block (:1284–1310) reading `totalFinished` , `thisMonthFinished` , `organizationRate` and `avgAttendance` . This is right for what it is — a club screen, not a season screen — but nothing on it says so.

evenings-22-on-one-screen-0-on-the-next (ROUND4 #2) is the collision of those two rows in the table. Hand-recomputed from the 23 raw documents, exactly as the code computes it:

```
lifetime: finished 22, cancelled 1, organizationRate 0.9565 (96%),
          activeThisMonth 7, activeThisYear 7,
          longestStreak 22 held by 1IdtNEjbEXfiRSqvLrJVn99Nsfi2,
          attendance { 22, 20, 20, 20, 20, 20, 19 }
season s2: finished 0, cancelled 0, organizationRate 0, activeThisYear 0,
          longestStreak 0, attendedByUser {}
```

So CommunityDetailsScreen prints "מפגשים שנערכו 22" and "אחוז הצלחה בארגון 96%", and one tap away CommunityStatsScreen prints "מחזוריים 0". Both are correct answers to different questions, neither screen states which question it asked, and the shipped 1.1.9 binary makes it worse: at 4de80d0 the organisation donut read the *season* rate (`stats?.organizationRate` , :876 of that revision) and rendered **0% in green** — "no evening was ever cancelled" and "every evening was cancelled" render identically.

`playerCompareService` — `compare-card-divides-season-goals-by-lifetime-nights` is **fixed at HEAD and unshipped**. The card's goals/assists/wins/rounds come from `getCommunityChampionship` (the rollup a close zeroes) while `games` is overridden from the scan (`:191-192 : games: attended[uidA] ?? baseA.games`), and `toPlayer` divides one by the other (`:103 : goalsPerGame = row.goals / row.games`). With an unscoped scan that is *this season's goals over the club's lifetime evenings*. On this club it produced goals 0-0, assists 0-0, 0.0-0.0 מחזור למחזור, ממוצע גולים 0%-0%, אחוז ניצחון 0%-0%, and then **22-20 מחזורים**, a single non-zero row, which set `verdict.leader` and printed "אתה מוביל 🏆 ב-1 מתוך 6 קטגוריות" ("you lead in 1 of 6 categories") over five zeros — on a card designed to be captured to PNG and shared. HEAD passes the season, which makes both columns read 0 and makes the new `if (!baseA || !baseB) return null guard ( :178 )` the thing that stops the card rendering at all. The club league table solved the same problem the other way: `CommunityChampionship.tsx:179` suppresses the override outright (`attendedByUser={seasonNo || seasonId || rows ? undefined : attendedByUser}`) rather than re-scoping it.

`assistantInsightsService` deliberately stays lifetime. Its two consumers are `rules.ts:425-438` : `attendanceStreak >= 3` → "הגעות רצופות N אתה מחזיק רצף של" and `attendedNights >= 10` → "N ערבים על הדשא". Both are statements about a person's history, not about a season, so the unscoped scan is the right source — but the *same object* also carries table-derived places and crowns from the season-zeroed rollup, so one coach card mixes a 22-night lifetime streak with a season standing of nothing. ROUND4 #6 described `getClubInsight` returning `null` before it could say either; HEAD splits the two cases (`:163 if (!mine && rows.length > 0) return null`) so the attendance half still speaks when the table is empty. The 200-document scan is still paid for, every 6 h per user, on a branch named `perf/firestore-read-costs` .

`SeasonsSettings` (`:339-351`) uses the scan for something else entirely: `st.totalFinished` is the club's *pre-seasons history*, the number the activation sheet promises will become season 1. Unscoped is correct — the club has no season yet — and the call is made only on the `firstTime` branch. For a live club the same state is filled from `seasons.playedRounds` instead (`:336`), which for this club is **0** while the scan would say 22.

The three scopes on the statistics screen

`scope` is a union (`:60`): `{k:'current'}` , `{k:'season', id}` , `{k:'all'}` . `slice ( :356 )` is `null` for the running season, the parsed archive for a closed one, and the merge for all-time; the whole screen derives from `viewChamp / slice` rather than from `champ` directly, so a past season renders through exactly the same code path as the running one.

Row on screen	running season	closed season	all time
tiles / גולים / בישולים / משחקונים	live rollup (<code>champ</code>)	archive <code>totals</code>	<code>mergeAllTime</code> totals
(<code>:769</code>) מחזורים tile	<code>season scan</code> <code>totalFinished</code>	<code>card</code> <code>scopedCard.completedRounds</code>	<code>lifetime.totalFinished</code>
top scorer hero, leaders card	live rollup rows	archive <code>players</code> rows	merged rows
<code>mostLoyal</code> / כתר ההתמדה (<code>:479-490</code>)	<code>scan</code> <code>topPlayers[0]</code>	archive <code>games</code> column	merged <code>games</code> column

Row on screen	running season	closed season	all time
הצמד הקטלני	live communityPairStats	archive pairs	not offered (pair data is per-season)
organisation donut (: 907–914)	lifetime	hidden	hidden
longest streak row (: 943–953)	lifetime	hidden	hidden
פעילים השנה (: 975–985)	lifetime activeThisYear	replaced by "N שחקנים שיחקו בעונה" from row count	same replacement
section כימיה	live pairs	hidden	hidden
league table (own fetch)	rollup with keepAll=true	archive	merged rows
club badges + level (: 494–517)	lifetime + archived goals	hidden	hidden

Three of those rows are lifetime *inside* the season scope, and the tooltip under the picker says so:

he.communityStatsScopeClosedInfo — "הכימיה בין השחקנים ותארי המועדון נמדדים על המועדון" — אחוז ההתארגנות, הרצפים, הכימיה בין השחקנים ותארי המועדון נמדדים על המועדון — "למרות כל הדרך, לא על עונה בודדת" — ולכן הם מוצגים רק בעונה הרצה between players and the club titles are measured over the club's whole life, not over a single season — which is why they are shown only in the running season"). The reason is that a record and a badge are permanent things the club did: season-scoping them meant a club un-earned its gold "שערי המועדון" badge and dropped a club level the morning after every close, and its 22-night attendance record vanished from the app entirely.

closed-season-tooltip-promises-lifetime is **true of HEAD and false of every shipped binary**. At 4de80d0 the donut read stats?.organizationRate, the streak row read stats.longestStreakUid / stats.longestStreak, and "פעילים השנה" read stats?.activeThisYear — all three season-scoped, all three zero, under a tooltip stating they were lifetime. Commit 5531a21 switched exactly those three to stats.lifetime.* and added the fields to make it possible. It also created ROUND4 #13: the **tile above them stayed season-scoped**, so the next build reads "מחזורי 0", then "96% מהמחזוריים שתוכננו יצאו לפועל", then "הגיע 22 מחזוריים ברצף", then "7 שחקנים היו פעילים השנה" — four rows, two scopes, one card, nothing marking the boundary. The old version was consistently wrong; this one is inconsistently right.

activeThisYear-is-season-intersected is why that switch was needed. The 365-day window is applied *inside* the post- inSeason branch (: 1213), so the season figure is the intersection of a year and a season. A season three days old makes "השנה" ("this year") mean three days, and this club — whose entire roster of seven played this month — read "0 שחקנים היו פעילים השנה". The invariant the two tallies exist to hold is season ≤ lifetime for every counter; it holds by construction, because the season branch is nested inside the lifetime one. The mock block at : 1034–1105 used to violate it (activeThisYear: 28 over lifetime.activeThisYear: 18), which is ROUND4 #25, and has been rewritten to derive both from one monotone tally.

appearances-rollup-undercounts-the-scan . mostLoyal deliberately reads the scan for the running season and the archive's games column for a closed one — and the two disagree. Production, side by side:

scan (attendedByUser)	archive seasonSummary/...s1 (players[].games)
1IdtNEjb... 22	1IdtNEjb... 19 ← mostLoyal winner, value 19
YIZlKWBv... 20	YIZlKWBv... 18
B5KpY04I... 20	B5KpY04I... 18
alsobLSA... 20	alsobLSA... 18
CEkRfDs2... 20	CEkRfDs2... 18
K5rSGB4J... 20	K5rSGB4J... 18
JoLFRxFr... 19	JoLFRxFr... 17

The gap is exactly three evenings — 2SMrLCGHyjnCmfRVthlS (06.07), MMtE8J6HCLLHpjhHL99A (16.07) and FCa5UtSdIb7m3X9rs9og (04.08) — and I confirmed the cause directly: those three games have an **empty finishCredited subcollection**, while every other night has one document in it. That marker is the idempotency latch created in the same batch as the `games: increment(1)` writes (`functions/src/index.ts:5895-5915`); no marker means the increment never ran, and §7 walks the credit block that writes it. So the scan is right and the rollup is short. The card for season 1 prints `completedRounds: 22` while the title it awarded, כתר ההתמודה, was decided on 19 — and the eligibility denominator for all nine titles was derived from the same 19 (`seasonRollover.ts:541-544`).

`archived-goals-silently-zero`. Lifetime club goals are assembled on the client as `champ.totalGoals + archivedGoals (:508)`, where `archivedGoals` is summed from `seasonHistoryService.list (:217-219)`. `list` returns the string `'error'` on failure and the handler is `if (!alive || list === 'error')` `return;` — so a failed read leaves `archivedGoals` at its initial `0` **and** leaves `pastSeasons` empty, which removes the "כל הזמנים" chip and every past-season chip from the picker. One failed `seasonCards` query silently collapses club goals to this season's (zero, after a close), and removes the only route to the club's history. On this club the damage is currently invisible: 27 archived goals against a 100-goal bronze threshold, and 303 vs 276 club-level points both landing in level 3. On a club with three seasons behind it the same failure is a visible demotion with no error shown.

What a close does to the live tables

`performSeasonClose` writes the archive, then winds the live counters back. `PLAYER_SEASON_FIELDS` (`functions/src/seasonRollover.ts:184-211`) names 19 columns — goals, assists, rounds, wins, losses, ties, games, cleanSheets, ownGoals, the six penalty counters, `csRounds`, `asRounds`, `eveningScoreSum`, `eveningScoreCount` — and every one is subtracted from each `communityPlayerStats` row. `PAIR_SEASON_FIELDS (:220-231)` does the same for all ten counters on every `communityPairStats` row. `CLUB_SEASON_FIELDS (:234-242)` — `rounds`, `goals`, `guestGoals`, `ownGoals`, `tiedRounds`, `shootoutRounds`, `scorelessRounds` — is **set to absolute 0** on `communityStats/{groupId}` (`:1004-1010`) rather than subtracted, which the module's own comment forbids for the other two.

The result, read live today, three weeks after the 17.09 close:

```
communityStats/HhzIwmjMl1i5HSOGht3p = { goals: 0, rounds: 0, guestGoals: 0,
                                             ownGoals: 0, tiedRounds: 0, ... }
all 7 communityPlayerStats rows      = { goals:0, assists:0, wins:0, games:0, rounds:0,
                                             cleanSheets:0, penTaken:0, ... }
```

Everything downstream of those rows is therefore zero at once. `rankChampionshipRows` (`src/utils/championship.ts:139–147`) keeps a row only if `keepAll || goals>0 || assists>0 || games>0 || wins>0`, and `getCommunityStats`'s sibling `getCommunityChampionship(groupId)` is called with no `memberIds` from the stats screen (`gameService.ts:1365`, `keepAll=false`) — so `champ.players` comes back **empty**, `hasScoring` is false, and the hero card, the leaders card and six of the eight fun facts disappear in one paint. The league table at the bottom of the same screen makes its *own* call **with** `memberIds` (`CommunityChampionship.tsx:90`, `keepAll=true`) and therefore renders seven rows of zeros: one screen, one function, two different answers to "does this club have players". The banner `he.communityStatsSeasonFresh` exists precisely to stop a ten-year member reading that as "— לי הכל" מחקו לי הכל"עונה N נמחק" ("season N has only just begun, so the table is still empty. Nothing was deleted").

How long does it last? Until the club's **next evening finishes with evidence of play**. That single transition (`didEveningHappen false→true`, `functions/src/index.ts:5870–5915`) writes `games: increment(1)` for every non-no-show member, which re-populates the rollout rows and so restores `champ.players`; the same evening is stamped `s2` by the trigger, which gives the season scan its first `totalFinished`. For a club that plays once a week that is **up to seven days of a club that looks brand new**, and for a club that has stopped playing it is permanent. Goals, assists, wins and mini-games do not come back with it — those are written only by the advanced-mode mini-game commit path, so the common timer-only club stays on `goals: 0` for the whole season, and `hasScoring` (`:564`) never becomes true. The "all time" scope is the only place where the club's real totals survive, and it is offered only once `pastSeasons.length > 0` (`:625`).

`mergeAllTime` (`src/utils/allTimeTable.ts`) is the addition: the live slice first, then every archive, summing thirteen count columns (`SUMMED`, `:35–49`) per uid and merging names first-wins so a current member keeps their live name over a frozen copy. Its one piece of care is `mergeCoverage` (`:63–69`): `csRounds` and `asRounds` are summed **only when both sides have one**, because absent means "never measured" and adding a present one to an absent one would invent coverage and deflate the rate. Everything else is a plain `n(prev[f]) + n(row[f])` with `n(undefined) = 0`, so an older archive missing a column contributes a confident zero — the `older-archive-degrades-into-plausible-zeros` finding, reached through the merge rather than through the reader.

10. The personal season summary

`SeasonSummaryScreen` (`src/screens/profile/SeasonSummaryScreen.tsx` , 951 lines) is where the end-of-season push lands every player who took part. It is titled "סיכום העונה" (*the season summary*) and it answers exactly one question: what happened to **me** between the season opening and its close. Nothing on it is cumulative — the footnote says so in as many words (`he.seasonSummaryFootnote` : "*all the numbers here are for this season only; the cumulative statistics are in the profile*").

Three files do the work. `src/services/seasonSummaryService.ts` (636 lines) fetches; `src/utils/seasonPersonal.ts` (427 lines) computes, and is pure; the screen and `src/components/summary/SeasonShareCard.tsx` (348 lines) render. The service is the only place that knows whether the numbers came from a live collection or from a frozen archive — `buildPersonalSeason` takes one shape either way, and the screen never learns which it got.

Two load paths

`seasonSummaryService.load()` (:416–539) takes `{ groupId, userId, seasonId? }`. The `seasonId` is omitted for "the season running now" and set for a specific one.

Path A — the sealed archive. If `seasonId` is given, the very first read is the archive (:432–446):

```
if (seasonId) {
  const archived = await getDoc(doc(db, 'seasonSummary', `${groupId}__${seasonId}`));
  if (archived.exists()) {
    return fromArchive(archived.data() as Record<string, unknown>, {
      groupId, userId, seasonId,
      seasons: await readSeasons(db, groupId), // may be undefined
    });
  }
}
```

Archive-first is deliberate and load-bearing: a player who has since **left the club** cannot read `/groups/{groupId}` at all, but the push about a season they played still points here. `readSeasons` (:322–335) swallows its own permission error and returns `undefined`, in which case `available` degrades to a one-entry picker holding only this season (:405–407). Everything else — the club's name, the final table, the pair counters, the titles — comes out of the one document. Cost: **two document reads**, one of which is optional.

Path B — the live tables. Falling through means the running season. It reads the club document for `seasons` (bailing with `null` if `seasons.enabled` is false), then three queries in parallel (:465–481):

```
const [statRows, asA, asB] = await Promise.all([
  getDocs(query(collection(db, 'communityPlayerStats'), where('groupId', '=', groupId))),
  getDocs(query(collection(db, 'communityPairStats'), where('groupId', '=', groupId), where('a', '=',
  getDocs(query(collection(db, 'communityPairStats'), where('groupId', '=', groupId), where('b', '=
]));
```

The pair collection is stored once per unordered pair under a sorted key, so "my pairs" is the union of the rows where I am `a` and the rows where I am `b`. On the one club that runs seasons (`HhzIwmjMl1i5HSOGHt3p`, מועדון (שכחת שושי) the collection holds **314 pair documents**, and for `1IdtNEjbEXfiRSqvLrJVn99Nsfi2` (מתן לוי, whose uid sorts first) the `a` query returns **39** and the `b` query **0** — re-run today against production, and a direct consequence of his uid beginning with a digit: every pair he is in is stored with him in the `a` slot. So one open of the live path costs 1 group doc + 7 stat docs + 39 pair docs + up to 6 `/users` reads $\approx$ **50 document reads, repeated on every pull-to-refresh**, against 2 for the archive. Many of those 39 pair rows are `guest:...` partners that `orient()` discards a moment later.

What differs between the two paths

	Archive (closed)	Live (running)
Player rows	<code>d.players</code> map, uid $\rightarrow$ row	<code>communityPlayerStats</code> query
Pair rows	<code>d.pairs</code> map	two <code>communityPairStats</code> queries
<code>completedRounds</code>	<code>d.completedRounds</code> (:382)	<code>seasons.playedRounds</code> (:512)
<code>groupName</code>	frozen in the archive	live <code>groups.name</code>
<code>endsAt</code>	<code>d.endsAt</code>	<code>null</code>
<code>roundsCadence</code>	never set	set iff <code>cadence.type === 'rounds'</code> (:515–523)
<code>myTitles</code> / <code>seasonTitles</code>	from <code>d.awards</code>	<code>always []</code> (:526–527)
Peer names	frozen <code>displayName</code> only	<code>/users</code> fetch
<code>peerAvatars</code>	<code>always {}</code> (:404)	avatar from the same <code>/users</code> fetch
Reads	2	$\sim$ 50

Two consequences follow from that table and both are visible to users. Titles do not exist for a running season by design — they are decided when the numbers stop — so the "תארים בעונה" card is gated on `me.hasData && model.closed` (screen :476). And the archive path deliberately makes no `/users` read, so **every peer row on a closed season draws the deterministic fallback disc**. That is the documented price of a summary that still renders after somebody deletes their account (:400–403), not a bug; the bug (`peer-rows-always-show-the-generic-auto-avatar`, P3) was on the *live* path, where the user document was already being fetched for

the name and the avatar thrown away. `resolvePeers` (:277–309) now keeps both — but note its `frozen` parameter is dead: the only call site passes `new Map()` (:496), because the archive path resolves its own names inline at :395–399 .

Every number on the screen

`buildPersonalSeason` (`seasonPersonal.ts`:307–427) reads **one row** — `players.find(r => r.userId === me)` — and derives the rest. Absent is zero throughout (`num()` , :147–148), which is the root of `older–archive–degrades–into–plausible–zeros` (P2): a season sealed before a counter existed renders a confident `0` rather than a dash. Only `csRounds / asRounds` are allowed to be absent, and `playerRow` (`seasonSummaryService.ts`:164–165) preserves that distinction on purpose.

Field	Derivation	Rendered as (<code>he.*</code>)
<code>goals / assists</code>	row fields	שערים / בישולים, lead tiles
<code>contributions</code>	<code>goals + assists</code> (:362)	"שערים + בישולים"
<code>rounds</code>	row <code>rounds</code> — mini-games (משחקונים)	"משחקונים"
<code>evenings</code>	row <code>games</code> — evenings (מחזורים)	"ערבי משחק"
<code>wins / losses / ties</code>	row fields	ניצחונות / הפסדים / תיקו
<code>winPct</code>	<code>wins / rounds</code> (:336 , :368)	"אחוז ניצחון"
<code>cleanSheets</code>	row field	"שערים נקיים"
<code>cleanSheetPct</code>	<code>cleanSheets / csRounds</code> (:340 , :370)	"אחוז שער נקי"
<code>goalsPerRound</code>	<code>goals / rounds</code> (:376)	"שערים למשחקון"
<code>assistsPerRound</code>	<code>assists / **asRounds**</code> (:377)	"בישולים למשחקון"
<code>ownGoals</code> , <code>penTaken/Scored/Faced/Save</code> <code>d</code>	row fields; tiles hidden at 0 (screen :535–553)	
<code>partialCoverage</code>	<code>rounds > 0 && (csRounds < rounds \\\ asRounds < rounds)</code> (:378)	the note at :564–566
<code>hasData</code>	<code>rounds > 0 \\\ games > 0</code> (:359)	gates the personal half

`rate()` (:155–157) returns `null` , **never 0** , for a zero denominator — "an unknown rate is not a zero one" — and the screen prints – for null (`pct / per` , screen :53–55).

The three denominators are three different numbers, and that is intentional. `winPct` divides by `rounds` ; `cleanSheetPct` by `csRounds` ; `assistsPerRound` by `asRounds` . Clean sheets have only been recorded since 17.08 and assists since 21.06, both long after clubs started counting mini-games, so for any veteran the three tiles in one grid genuinely cannot be reconciled by arithmetic. The production archive proves it: מתן לוי has

rounds: 26, csRounds: 15, asRounds: 24. His grid reads 12 אחוז שער נקי, 80% (12/15, not 12/26 = 46%), 1 (1/24) בישולים למשחקון 0.04. בישול, partialCoverage is true for **all seven players** of s1 — every one has csRounds < rounds — so the explanatory note (he.seasonPartialCoverageNote) renders for the whole club.

Two honest criticisms of this design. First, winPct = wins / rounds counts ties in the denominator, which is the weaker statistic; the code says so at :327–335 and keeps it only because the club's efficiency table has shipped for months computing the same thing under the same Hebrew label. Second, assistsPerRound dividing by asRounds is the fix for assists-per-round-denominator-disagrees-with-club-efficiency-tab (P2) — the shipped binary divides by rounds (see the last sub-section), which gave four of the seven s1 players two different "בישולים למשחקון" one tap apart. **There is no test for it.**

tests/logic/seasonPersonal.test.ts (411 lines, 29 cases) has a case for the clean-sheet denominator and not one mention of asRounds, assistsPerRound or partialCoverage.

WORKED EXAMPLE — מתן לוי, SEASON 1, FROM THE LIVE ARCHIVE BYTES

```
row: { goals:2, assists:1, rounds:26, games:19, wins:11, losses:9, ties:6,
      cleanSheets:12, csRounds:15, asRounds:24, penTaken:1, penScored:0,
      penFaced:1, penSaved:1, penMissed:1, ownGoals:0 }
```

renders: שער 2 · בישולים 1 · שערים+בישולים 3 · ערבי משחק 19 · משחקונים 26 · ניצחונות 11 · הפסדים 9 · תיקו 6 · אחוז ניצחון 42% · שערים נקיים 12 · אחוז שער נקי 80% · שערים למשחקון 0.08 · בישולים למשחקון 0.04 · פנדלים שהובקעו 0/1 · פנדלים שנעצרו 1/1. Note $11 + 9 + 6 = 26 = \text{rounds}$ exactly — the win/loss/tie triple does close over the mini-game count on all seven rows, which is the one invariant here that demonstrably holds.

The three ranks

rankOf (:258–297) answers "what position does the club table put me in", not "how many are ahead of me". Those differ whenever ties are shared, and the difference was user-reported ("המיקום שלי ... לא נכון").

```
const played = rows.filter((r) => (num(r.rounds) > 0 || num(r.games) > 0) && isReal(r.userId));
const order = [...played].sort((a, b) =>
  pick(b) - pick(a) ||
  num(b.wins) - num(a.wins) || num(b.goals) - num(a.goals) ||
  num(b.assists) - num(a.assists) || a.userId.localeCompare(b.userId));
const i = order.findIndex((r) => r.userId === me);
const anyValue = order.some((r) => pick(r) > 0);
return { rank: i < 0 || !anyValue ? null : i + 1, of: order.length };
```

Three things to read out of it. of is "who turned up" — rounds > 0 || games > 0, the same test hasData makes, and isReal (:152) drops guest: identities, who are a different person each week. That || games > 0 is the fix for rank-denominator-of-excludes-attendees (P1): the shipped code filters on rounds > 0 alone, so a timer-only club — which records **zero** mini-games, ever — gives every player of: 0 and dashes out

seven of the thirteen tiles while the club table one tap away lists everybody. 23 of the 93

`communityPlayerStats` rows in production have `games > 0` and `rounds === 0`; those are exactly the players the shipped denominator erases.

The `anyValue guard` is the fix for a regression the widening itself caused: with `of` now counting attendees, a column in which every player has 0 still produced a sort, which fell all the way through to

`userId.localeCompare` and handed three players a **gold medal** for "מקום 1 מתוך 15 בשערים" in a season with no goals in it. A null rank draws no medal (`rankTint / rankIcon` , screen :119–125) and prints `–` . It fires only when the *whole* column is zero, though: a player with 0 assists in a column where somebody has 5 is still given an ordinal position rather than a blank.

The tie-break mirrors the club table's comparator — and this is where the open P0 lives. `rankOf` breaks ties on wins → goals → assists → uid. `CommunityStatsTable`

(`src/components/community/CommunityStatsTable.tsx:153–156`) sorts on **one key with no tie-break at all**:

```
return [...effPlayers]
  .sort((a, b) => Number(b[sortKey as keyof ChampionshipRow] ?? 0) -
    Number(a[sortKey as keyof ChampionshipRow] ?? 0))
  .slice(0, limit);
```

V8's sort is stable, so ties keep the *incoming* order. For a sealed season that order is whatever

`parseSeasonTable` produced, and at HEAD that is `players.sort((a,b) =>`

`a.uid.localeCompare(b.uid))` (`src/utils/seasonArchive.ts:114`) — a uid sort. So the two surfaces

break ties on **different** keys, and the production archive contains a case where they disagree. Assists in s1 are

`{Nofar 5, Lioz 3, Linoy 2, 1 הלין ,1 איציק ,1 מתן , Eliran 0}`. `rankOf` breaks the three-way tie at 1 on wins first (הלין has 12, the other two 11) and reports **4 הליןth, 5 מתןth, 6 איציקth**. The club table, sorting stably over a uid-ordered input (`1Idt...` , `JoLF...` , `also...`), lists **4 מתןth, 5 איציקth, 6 הליןth**. Same document, same season, two screens, two answers — and the summary's answer is the one that decides whether a bronze medal is drawn and whether the share card prints a placing.

`sealed-season-rank-vs-club-table-nondeterministic` (P0) is worse than that in the **shipped** binary, where `seasonArchive.ts` has no sort at all and the incoming order is Firestore's map iteration order — an earlier audit round reported two consecutive reads of the immutable s1 archive coming back in different key orders, and although the two orderings were not recorded (treat the observation as asserted, not demonstrated), nothing in the protocol promises an order either way, so the silver ring on the club table is not guaranteed to survive a refresh. The HEAD uid sort makes the table *deterministic*; it does not make it *agree*.

The six peer rows

All six come from `communityPairStats` , which the close zeroes along with everything else — so the peer rows are season-scoped for free, with no new counter. How that collection is *filled* is the cause of the gap two

paragraphs below, and it is not what it looks like: only the legacy undirected `assists` field is written per mini-game, by `commitRoundStats` (`functions/src/index.ts:14474`). The nine counters these rows actually use

— `sameTeam` , `against` , `winsTogether` , `lossesTogether` , `cleanSheetsTogether` , `winsA` , `winsB` ,

`assistsAToB` , `assistsBToA` — are written **once per evening** by `rollUpClubPairs` (`index.ts:4695`), which re-reads the evening's mini-games from the `games/{id}/roundHistory` subcollection. Two writers, two windows; see §7.

Every read goes through `orient()` (`:181–201`), because a pair document is stored once under a sorted key and which slot I occupy is an accident of my user id:

```
const meIsA = row.a === me;
...
myWins:      meIsA ? num(row.winsA)      : num(row.winsB),
theirWins:   meIsA ? num(row.winsB)      : num(row.winsA),
iAssistedThem: meIsA ? num(row.assistsAToB) : num(row.assistsBToA),
theyAssistedMe: meIsA ? num(row.assistsBToA) : num(row.assistsAToB),
```

Getting that backwards is invisible — the screen still renders, it just names the wrong person as the one you beat all season. `orient` also returns `null` for a guest partner and for a malformed self-pair, which would otherwise make every player their own best teammate. It was verified over all 21 production pairs and is sound.

`best()` (`:211–241`) picks the maximum by `pick` , **excluding zero** (`if (count <= 0) continue`), tie-breaking on a per-row `tiebreak` and then on `o.other < winner.other` — a total order, so the same season always names the same person. The six rows:

Row	Hebrew label	pick	tie-break	detail line shows
partner	הכי הרבה יחד באותה קבוצה	sameTeam	winsTogether	count + winsTogether
nemesis	היריב הכי גדול	against	theirWins	count , myWins vs theirWins
victim	מי הובס הכי הרבה	myWins	against	count only
tormentor	מי ניצח הכי הרבה	theirWins	against	count only
assistedMost	למי הכי הרבה בישולים	iAssistedThem	sameTeam	count
assistedBy	ממי הכי הרבה בישולים	theyAssistedMe	sameTeam	count

`partner` 's `winsTogether` tie-break is deliberate — of two people you played 10 mini-games beside, the one you kept winning with is the better story. `nemesis` leans the other way, towards whoever beat you more.

De-duplication (`:402–423`). The three rival rows were picked independently, and in a club of regulars the person you face most is usually also the one you beat most and lose to most. On s1 that produced the same name and the same face three times for **5 of the 7 players**, and one of them saw the same person in all four rows he had (`nemesis-victim-tormentor-not-duplicated` , P1). The fix runs the three picks in order behind a `taken` set:

```
const peer = best(oriented.filter((o) => !taken.has(o.other)), pick, tiebreak)
    ?? best(oriented, pick, tiebreak);
if (peer) taken.add(peer.userId);
```

The fallback to the un-filtered `best` means a player with a single opponent still gets three true rows rather than two blanks.

△ **The de-duplication makes two of the three labels false.** Rows 2 and 3 no longer name the maximum — they name the best remaining candidate — while the Hebrew label still claims a superlative. Production, Linoy Levi (`CEkRfDs20x...`), s1, HEAD code:

- `nemesis` → Lioz Madar, detail "7 4 מול זה — 3 ניצחונו מול 4" (he beat her 4 times);
- `tormentor`, labelled "מי ניצח הכי הרבה" (*who beat me the most*) → מתן לוי, detail "2 הפסדים".

Lioz beat her 4, מתן beat her 2, and both numbers are printed on the same card two rows apart. The old bug produced a repetitive card; the new one produces a self-contradicting one. Neither the label nor the detail line carries any hedge, and the test suite's two dedup cases (`:353` , `:368`) assert only that the rows name different people, never that the row is still true.

△ **The assist rows describe half the season.** `assistsAToB + assistsBToA` summed over all 21 archived pairs is 7. `totals.assists` on the same document is 13, and the seven player rows also sum to 13. So six of the season's thirteen assists are attributed to no pair at all (`assist-peers-account-for-half-the-season-assists` , P2). §7 identifies the cause: the directional counters can only see assists that reached a `roundHistory` document, and the club's two oldest evenings predate that subcollection entirely — seven assisted goals are recoverable from `roundHistory` across all 23 games, which is exactly the 7 in the archive. The visible consequence: Nofar Tzabari won כתר הבישולים with `value: 5` , and one card below, her "למי הכי הרבה" row reads `אִיצִיק לוי · 2 בִּישׁוּלִים`. Three of her five assists are not in the pair data. The legacy undirected `assists` field on the same pair rows sums to 8 — a third number — which is why `archive-duo-reader-has-no-floor` matters: the duo title reads the directional fields and the live scope reads the legacy one.

When both `partner` and `nemesis` are null the card prints `he.seasonPeersEmpty` (screen `:658–662`), whose copy now names the cause — "*who played with whom is counted only in evenings run from the advanced live screen*" — rather than the old "it will fill itself up", which is a promise a timer-only club can never keep (`peers-empty-promises-it-will-fill` , P2).

Titles, champions, and the "did not play" case

`titlesFor` (`:195–210`) walks the nine `SEASON_TITLE_KEYS` and keeps the ones whose `winners` array contains me, computing `sharedWith = holders.length - 1`. The duo title is keyed `a__b` , so membership is tested against `w.split('__')`. `allTitlesOf` (`:219–243`) does the same for the whole club, resolving each winner through the archive's `frozen displayName map` (`'-'` when absent), and flags `mine` .

s1's awards, live: `topScorer` 10 הלקן צברי, `topAssister` Nofar 5, `topWinner` Lioz 13, `mostLoyal` 19 מתן, `cleanSheetKing` 13 הלקן, `penaltyKing` 1.0 הלקן, `penaltyKeeper` 1.0 הלקן, `deadlyDuo` null, `mvp` all seven players at 6. So the champions card lists eight rows (the null duo is skipped at `:228`) and every single player holds כתר העונה with `sharedWith: 6` .

The `hasData` split (screen :463–512) is the fix for `did-not-play-told-the-season-had-no-games` (P2). One ternary used to gate both halves, so a player who missed the season got a single card reading "לא שיחקת בעונה הזאת" **and lost the champions list with it** — the list that is identical for every reader, is the reason the close push sends the whole club here, and was fetched and thrown away on the way to the render. Now the personal cards stay behind `me.hasData` and the club's do not; the empty line (:491–499) explains the gap and the champions render underneath it. The two strings are distinct: `seasonSummaryNoRounds` ("you haven't played this season yet, so there's nothing to summarise") for a running season, `seasonSummaryNoRoundsClosed` ("you didn't play this season, so you have no summary from it") for a closed one.

`completedRounds` **is the season's EVENINGS**, and it is the note under the standing card: `he.seasonClubRounds(22) → "22 מחזורים שוחקו במועדון בעונה הזאת. המיקום מחושב מול מי ששיחק בה 22"`. The archive has carried `completedRounds: 22` since the archive existed and it was **read by nothing** (`archive-completedrounds-never-read-by-client`, P2); the shipped client substitutes `totals.rounds`, the mini-game count, so the same sealed season reads **22** on the history screen and **37** here, and a timer-only club reads "משחקים שוחקו 0" for a season it played every week of.

SeasonShareCard — what leaves the app as a PNG

The card is rendered off-screen at a fixed `SHARE_CARD_WIDTH = 340` (:30) inside a positioned stage (screen :691–696; positioned rather than hidden, because a `display:none` subtree has no layout and captures blank), and `onShare` (screen :330–362) turns it into a PNG with `captureRef` and hands it to `expo-sharing`. `allowFontScaling={false}` is set on every text node: the capture happens before anyone can see it, so at Android "Largest" a **100%** used to grow out of its tile and a 30-character name clipped mid-word.

Its contents, three tiles to a row (`PER_ROW = 3`):

```
const mini = me.rounds > 0;
const tiles = [
  { value: String(me.goals),    label: he.statGoals },          // שערים
  { value: String(me.assists),  label: he.statAssists },        // בישולים
  { value: String(me.evenings), label: he.seasonStatEvenings }, // ערבי משחק
  ...(mini ? [ rounds, winPct, cleanSheets, wins, losses, ties ] : []),
];
```

`evenings` first and never conditional; everything after it is a mini-game counter, printed only by a club that has any. That is the fix for `share-card-is-all-zeros-for-a-timer-only-club` (P1): the shipped card prints all eight mini-game numbers unconditionally and omits `evenings` entirely, so 23 of the 93 production stat rows produce a card of eight zeros and a dash to send to the group chat, while the screen behind it says they turned up fourteen times. Even at HEAD a timer-only player's card is `0 · 0 · N` — better, but two of the three headline numbers are still zero.

The rank gate (:95–99): `bestRank` is the numerically lowest of the three ranks, and

```
const worthShowing = !!bestRank && me.ranks.of > 1 &&
  ((bestRank.rank <= 3 && me.ranks.of >= MIN_CLUB_FOR_PODIUM) || bestRank.rank / me.ranks.of <= 0
```

MIN_CLUB_FOR_PODIUM = 8 . The shipped gate is `bestRank.rank <= 3 || rank/of <= 0.34` with no club-size floor, which printed "מקום 3 מתוך 3" — last place — in bold on a card about to be sent (`share-card-rank-gate-still-prints-last-place` , P2). One side-effect of the floor is worth naming: on a 7-player club, the podium arm can never fire, so only ranks 1 and 2 (0.14 and 0.29) clear the top-third arm. לוי's best placing in s1 is **3rd in wins of 7** — 0.43 — so his card carries no placing line at all.

Titles on the card print `{name} · {value}` plus, now, `· במשותף עם עוד N ( 209–211: )` `share-card-claims-a-shared-title-as-its-own` (P1) is that `sharedWith` was computed all along and dropped here: in the shipped binary all seven holders of s1's `כתר העונה` send the same exclusive-looking "ציון · `כתר העונה` 6.0" into the same group chat within a minute of each other.

Two peer lines — partner and nemesis only (`:220–235`), using `he.seasonSharePartner` / `seasonShareNemesis` , which were rewritten to say `משחקונים` rather than `משחקים` (the card prints "40 משחקונים" above, and calling the same unit two things is `share-card-names-one-unit-two-ways` , P3) and to drop the parentheses, because bidi resolves a closing paren after a Latin name to RTL and mirrors it — four of the seven frozen s1 names are Latin, and the PNG that left the app read "משחקונים 6" .

A complete HEAD card for לוי, s1: brand row · `1 עונה` pill · his name · מועדון שכחת שושי · tiles `2 / 1 / 19` then `26 / 42% / 12` then `11 / 9 / 6` · no placing line · `במשותף · ציון 6.0` · `כתר העונה · כתרי מחזוריים · 19` · `כתר ההתמודדה · 19 מחזוריים` · `כתר העונה · ציון 6.0` · `במשותף` · `משחקונים · היריב הגדול: איציק לוי · 10 משחקונים · 10 Lioz Madar · עם עוד 6 משחקונים · הכי הרבה יחד`

⚠ What the shipped binary actually does instead

Every fix described in this section landed after `4de80d0` and exists only in git (front matter, *What is running where*). `git diff --stat 4de80d0 HEAD: seasonPersonal.ts +92/−10, seasonSummaryService.ts +88/−21, SeasonShareCard.tsx +97/−29`. What users are actually running today:

```
// 4de80d0:src/utils/seasonPersonal.ts
:246 const played = rows.filter((r) => num(r.rounds) > 0 && isReal(r.userId)); // no `|| games`
:331 assistsPerRound: rate(assists, rounds), // not asRounds
:339 nemesis: best(oriented, (o) => o.against, (o) => o.theirWins), // no dedup
:340 victim: best(oriented, (o) => o.myWins, (o) => o.against),
:341 tormentor: best(oriented, (o) => o.theirWins, (o) => o.against),
// 4de80d0:src/services/seasonSummaryService.ts
:341 completedRounds: num(totals.rounds), // 37, not the archive's 22
:461 completedRounds: num(clubSnap.data()?.rounds), // an extra read of communityStats
```

Read against production s1 that means the live app tells לוי his three rivals are all `מי` — including `מי` "ניצח הכי הרבה: איציק לוי" about a man he beat 5–3 — divides his one assist by 26 instead of 24, and captions the club's season "37" where the hall of fame says "22". An expert reading HEAD is reading a version no user has.

11. The screens, and what each one reads

⚠ Before you compare this to the app on your phone

Almost nothing described below is in a store binary. Both 1.1.9 artefacts were built from 4de80d0 and every seasons UI commit landed after it; the commit list and the diff are in the front matter under *What is running where*. This is ROUND4 finding #1, and it colours everything in this section: where a defect below is marked *shipped*, a user can see it today; where it is marked *HEAD only*, it is a defect waiting for a build. Five files described here do not exist at all in the shipped binary — seasonCardVariant.ts, seasonMedalTier.ts, seasonScope.ts, SeasonMedal.tsx, SeasonPoster.tsx.

The shipped hall of fame is a different screen: 231 lines, a plain white card per season with a title row, a date line, he.seasonHistoryLine(22, 37, 7) → "22 מחזורים · 37 משחקונים · 7 שחקנים", and nine flat rows. No poster, no medals, no tiers, no streak badges, no void ribbon, no variant. Its titles are still מלך (topScorer: 'השערים', mvp: 'שחקן העונה' at 4de80d0:src/i18n/he.ts:2148–2156); HEAD renamed six of them to כתר.

The one club with real data

Everything below is worked against HhzIwmjMl1i5HS0GHt3p (מועדון שכחת שושי), the only club that has ever run a season. Read live:

source	value
groups/.../seasons	{enabled:true, currentId:'s2', currentNo:2, count:1, playedRounds:0, roundsAtStart:10, startedAt:1789679141952, reopenedAt:1789727847680, cadence:{type:'rounds', targetRounds:24}}
seasonCards/...__s1	{no:1, startsAt:1789547794676 (2026-09-16T08:36Z), endsAt:1789661526768 (2026-09-17T16:12Z), completedRounds:22, totals:{rounds:37, goals:27, assists:13}, players:7, winners:[8]} — no endedEarly, no partialData, no awardsDenominator
seasonSummary/...__s1	7 player rows, games ∈ {19,18,18,18,18,18,17}, awards.deadlyDuo: null, originalTarget.targetRounds: 22, roundsAtStartOfSeason: 7
all 23 terminal games	3 stamped s1, 1 stamped s3 (an orphan — the club has never had an s3), 19 unstamped, 0 stamped s2
all 7 communityPlayerStats	every counter 0 — the close zeroed them

The decision functions

seasonCardVariant(s) — src/utils/seasonCardVariant.ts:25–57. Sorts a finished season into 'full' | 'noTitles' | 'void':

```
const nothingRecorded =
  s.players === 0 && s.winners.length === 0 && s.completedRounds === 0 &&
  s.totals.rounds === 0 && s.totals.goals === 0 && s.totals.assists === 0;
if (nothingRecorded) return 'void';
return s.winners.length === 0 ? 'noTitles' : 'full';
```

It is deliberately **not** keyed on `totals.rounds` alone (mini-games are advanced-mode only) and no longer on `endsAt - startsAt` — on `s1` that span is 31.6 hours for three months of football, so a 48h "void" window would have swallowed the club's only real season. Ledger finding *variant-tests-pass-against-the-exact-regression-the-file-warns-about* stands: all five cases in `tests/logic/seasonCardVariant.test.ts` also pass against an implementation keyed on `totals.rounds`, so the test suite does not defend the property the file's comment is about.

`heroSeasonId(list)` — :65–70. The season the list opens on and the only one that gets confetti: first 'full', else first 'noTitles', else `null`. For this club → 's1'.

`heroWinner(s)` — :79–101. `HERO_ORDER = ['mvp', 'topScorer', 'topWinner']`, take the first whose `names.length <= 2`; otherwise fall back to the title with the **fewest** holders. On `s1`, `mvp` has seven names, so it is skipped and `topScorer` (הלן צברי, 10) becomes the poster subject.

`medalTier(key, value, completedRounds)` — `src/utils/seasonMedalTier.ts:103–123`, with per-title scales at :57–67. Counting titles have their own three-stop `steps`; the two penalty titles use `RATE_STEPS = [0.7, 0.85, 1]`; `mvp` uses `MVP_STEPS = [6.6, 7.6, 8.8]`; `mostLoyal` divides by the season's own length against `[0.6, 0.8, 1]`. `step()` compares with `>=`.

`titleStreak(seasons, index, key)` — :141–177. Walks towards older seasons while the season numbers stay consecutive and the holder set is identical. **It compares by NAME**, because a `SeasonWinner` is `{key, names, value}` and no uid survives the archive (the file says so). A player who renames themselves silently ends their own streak, and nothing can distinguish that from a new holder. The names are joined on `U+0000` (a literal NUL) after `trim()` and `sort()`, which closes the older `['ג', 'ב', 'א']` vs `['ג', 'ב', 'א']` collision.

SeasonsCard — the club screen

`src/components/community/SeasonsCard.tsx`, mounted at `CommunityDetailsScreen.tsx:902` with `{groupId, seasons: group.seasons, isMember: isMember || isAdmin}`. It reads **only** the `seasons` block of the group document already in memory — no query, no read.

Three states:

1. `!isMember` → `null` (:55). Every collection behind its buttons is membership-gated.
2. `seasons.enabled !== true` and `count > 0` (:62–77) → a stub card: `he.seasonsOffButArchived` plus the "עונות קודמות ותארים" button. `enabled !== true` and `count === 0` → `null`.
3. Running (:132–187).

The running card's fields:

pixel	source
עונות • עונה 2	he.seasonsCardTitle , he.seasonNumberLabel(seasons.currentNo ?? 1) (:137)
①	he.seasonsCardInfo (:143) — the only place in the app that defines מחזור for the reader
big number 0	seasons.playedRounds ?? 0 (:151) — rounds cadence only
מתוך 24 מחזורים	he.seasonsCardOfTarget(cadence.targetRounds) (:153)
progress bar	playedRounds / targetRounds , clamped (:118–127); turns colors.success at ≥ 1
נשארו 24 מחזורים	he.seasonsCardRemaining(max(0, target – played)) (:87)
days-left line	date cadence only, from endsOn or endsAt (:104–113)
history button	shown iff seasons.count > 0 (:177)

For a date-cadence season, `progress` is derived from `startedAt` and `daysLeft` rather than from the cadence (:123–126), so it is only as honest as `startedAt` — and `startedAt` is stamped when seasons are **switched on**, not at the season's first evening. On this club `startedAt` = 1789679141952 = 2026-09-17T21:05Z, five hours after s1 closed and a day before the reopen at 2026-09-18T10:37Z.

Today this card renders 0 / מתוך 24 מחזורים , an empty bar and נשארו 24 מחזורים — which is correct for s2 and sits one tap away from a stats screen that will tell you the club has played 22 evenings. The ledger's *card-promises-an-evening-of-grace-that-is-an-hour* is fixed: the at-target string is now 'העונה הגיעה ליעד ותיסגר'. מעצמה בשעה הקרובה.

SeasonsSettings — the admin controls

`src/components/community/SeasonsSettings.tsx` , 1,150 lines, mounted as `extraAdvanced` inside the club-edit form (`CommunityEditScreen.tsx:181`). It is the only surface that writes anything.

State seeding (:230–306) is all derived from the club document: `live = seasons.enabled === true ; cadence from seasons.cadence.type ; months / rounds from the stored target whenever valid`, not only when it matches a chip (:253–262); `customMonths / customRounds` open pre-expanded when the stored value is off-preset. `firstTime = (count ?? 0) === 0 && !live ( :309 )`. `thisSeasonNo = live ? currentNo : count + 1 ( :313 )` — numbering continues across a disable/enable, which is why every string here is parameterised.

`history ( :314–355 )` is the number every validation and every confirmation line is measured against, and it comes from **two different places**:

```
if (!firstTime) {
  setHistory(live ? Math.max(0, seasons?.playedRounds ?? 0) : 0);
  return;
}
// firstTime only:
gameService.getCommunityStats(groupId).then(st => setHistory(st?.totalFinished))
```

For a live club that is `seasons.playedRounds` — the mirror, `0` today. For a first activation it is the **unscoped** 200-document evening scan — 22 for this club. A failed read sets `historyFailed` and leaves `history` at `null` rather than `0`, and every button below is disabled while it is `null` (`:944–947`); `0` would have told the confirmation sheet the club had never played, one press before the server sealed 22 evenings.

`plan` (`:419–441`) is one `planActivation(...)` object that drives the red line, the button's enabled state and every row of the sheet. `planErrorText` (`:171–195`) maps its six refusal codes onto Hebrew that names the actual season number — `he.seasonsErrSeasonEndOf(no)` etc. The ledger's P0 *live-club-cannot-use-date-cadence* is fixed at HEAD by `hasHistory` (`seasonActivation.ts:160–171`), which derives the end date itself instead of demanding a control that only renders under `firstTime`; ROUND4 #21 (that branch returning no `nextStartsOn`) is fixed in the same place. **Both fixes are HEAD only** — a club on season 2 running 1.1.9 still cannot choose a date cadence.

Controls, top to bottom: the toggle with its `InfoTip` (`:637–667`); `he.seasonsCadenceQuestion` and two chips; length chips `MONTH_CHOICES = [3,6,12]` / `ROUND_CHOICES = [24,48,96]` plus a `מותאם אישית` chip opening a `Stepper` (`ROUND_STEP` is now `1`, closing the ledger's *rounds-stepper-lattice*); a three-line date preview for a non-first-time date club (`:770–790`); the `firstTime`-only history block with `לצרף לעונה` `לסגור ולהתחיל מאפס` / `הנוכחית` and, under `cadence==='date' && !sealHistory`, the season-1 end-date chips built from `SEASON1_MONTH_CHOICES = [1,2,3,6]`.

Three destructive buttons:

- `הפעל עונות` opens `SeasonConfirmSheet` (`:990–1010`); only the sheet's own button calls `seasonService.enable`.
- `עדכן את יעד העונה` (`saveTarget`, `:513–523`) is a **bare one-tap write** — no sheet, no confirmation. `targetChanged` (`:401–408`) compares only `c.months !== months` for a date club, so the ledger's *date-season-cannot-be-renewed-at-the-same-length* still holds: a date club that wants the same length again cannot press the button at all.
- `סיים עונה עכשיו` (`endNow`, `:590–620`) builds its body from `seasons.playedRounds`. For this club today that is `he.seasonsConfirmSealedNow(0) → "עונה 2 · 0 מחזורים קיימים – תיסגר כעת"`, followed by `he.seasonsEndConfirmBody`. The server's own comment on `endSeasonNow` claims this client shows the admin which titles are about to be awarded; it does not — the titles are not computed anywhere this component can reach.
- `בטל את סגירת העונה האחרונה` (`reopenLast`, `:525–588`) now reads the season's own card **before** opening the dialog, and only warns about an immediate re-close when `!last.endedEarly || last.completedRounds >= target`. It is gated on `live && count > 0` (`:979`), which closes the ledger's case of it sitting next to "הפעל עונות" on a seasons-off club. It still sits directly beneath the red end-season button, same size, same shape.

`SeasonConfirmSheet` (`src/components/community/SeasonConfirmSheet.tsx`) renders one `Row` per present field of the plan, so a plan missing a field silently drops a line — which is exactly how ROUND4 #21 manifested.

CommunityStatsScreen and its scope picker

`src/screens/communities/CommunityStatsScreen.tsx`, 1,439 lines. `type Scope = {k:'current'} | {k:'all'} | {k:'season', id} (:60).`

The load order matters (:182–197): the **group document is read first**, because the evening scan has to know which season it is counting.

```
const scope = g?.seasons?.enabled && g.seasons.currentId
  ? { currentId: g.seasons.currentId, currentNo: g.seasons.currentNo ?? 1 }
  : undefined;
gameService.getCommunityStats(groupId, scope)
```

inSeason (src/utils/seasonScope.ts:28–36) is stamp ? stamp === season.currentId : season.currentNo === 1 . On this club currentNo is 2 and **no game carries** s2 , so the season-scoped half of getCommunityStats returns totalFinished 0 , organizationRate 0 , activeThisYear 0 , longestStreak 0 , topPlayers [] , while the lifetime block returns 22 / 96% / 7 / streak 22 held by 1IdtNEjb... .

Which of the two each pixel reads, at HEAD:

row	line	reads
hero tile מחזורים	:769	scopedCard ? scopedCard.completedRounds : scope==='all' ? lifetime.totalFinished : stats.totalFinished → season-scoped, 0
organisation donut	:908–913	lifetime.organizationRate → 96%, gated on lifetime.totalFinished + totalCancelled > 0
longest streak	:943–951	lifetime.longestStreak → 22, name from lifetime.longestStreakUid
פעילים השנה	:981	lifetime.activeThisYear → 7
club badges + level	:500–514	lifetime.totalFinished , champ.totalGoals + archivedGoals (0 + 27)
everything else	deriv ed	the slice / champ rollup, which a close zeroes

That row-by-row split is ROUND4 #13, and it is HEAD only: the next build will read 0 מחזורים , then 96% שחקנים היו פעילים ברצף – 22 מחזורים , then הגיע 7 שחקנים היו פעילים – 22 מחזורים , then מכל המחזורים שתוכננו במועדון אי פעם , on one card. The shipped 1.1.9 reads 0 / 0% / no streak row / 0 שחקנים — ROUND4 #2, the bug that started the audit, live on phones right now. ROUND4 #12 (the name prefetch still asking for the season-scoped uid) is fixed at :279–280 , which now adds both uids.

The picker itself (:604–641) renders when seasons.enabled || count > 0 , and the chip row only when count > 0 . Chips: עכשיו N עונה , then כל הזמנים (only when pastSeasons.length > 0) , then one per closed season. Selecting a past season fetches seasonSummary/{groupId}__{seasonId} through seasonHistoryService.table ; כל הזמנים fetches every archive and mergeAllTime s them with the live rows, and bails to {k:'current'} if any archive is missing (:316). scopeLoading (:375) forces viewChamp to null while the archive is in flight, so the running season's numbers can never render under a closed season's heading.

Two things the picker deliberately hides outside `scope==='current'`: the chemistry section (:866) and the club achievements (:1035). Two things it deliberately shows only outside it: "7 שחקנים שיצקו בעונה" from `derived.players.length` (:958) and the `N` סיכום עונה button (:714–727).

SeasonHistoryScreen, SeasonPoster, SeasonMedal — the hall of fame

`src/screens/communities/SeasonHistoryScreen.tsx` (700 lines at HEAD, 231 shipped). Data: `seasonHistoryService.list(groupId) → getDocs(query(collection(db, 'seasonCards'), where('groupId', '==', groupId)))`, sorted `b.no - a.no`. It reads the **cards**, not the archives — a five-season sixty-player set of archives is over a megabyte and this screen needs a date, three numbers and nine names.

State machine (:294–343): `seasons === null → SoccerBallLoader`. A failed **refresh** sets `failed` and returns without touching `seasons`, so the list survives; a failed **first** load additionally sets `seasons` to `[]`. `failed && seasons.length > 0` renders the amber `he.seasonHistoryRefreshFailed` above the list; `failed && empty` renders `he.seasonHistoryLoadFailed`; not-failed and empty renders `he.seasonHistoryEmpty`. `viewLogged` (:311) makes `SeasonHistoryViewed` fire once per visit rather than once per pull. The list is a `FlatList` with `initialNumToRender: 2` — twelve sealed seasons in a plain `ScrollView` meant ~228 SVG roots and ~336 gradients in one frame.

Header crest (:438–463): `he.seasonHallTitle = היכל התהילה`, `he.seasonHallClosed(1) = עונה סגורה`, then `totals.rounds = Σ completedRounds = 22 מחזורים` and `totals.mini = Σ totals.rounds = 37 משחקונים` (the mini tile is suppressed at 0).

Worked example — what s1 renders today, field by field. `seasonCardVariant(s1) = 'full'` (players 7, winners 8). `heroSeasonId = 's1' → celebrate = true → ConfettiBurst + LightSweep` on every open. `heroWinner` skips `mvp` (7 names) and returns `topScorer`.

`SeasonPoster` (`src/components/community/SeasonPoster.tsx`):

- ghost numeral 1 at 168pt, `season.no` (:113)
- tint `seasonTitleTint('topScorer') = colors.primary`; ghost icon `football` at 84pt
- pill `כתר השערים` (`he.seasonTitleNames.topScorer`)
- hero name `הלן צברי` at 35pt, capped at `MAX_HERO_NAMES = 2`
- value row: `he.seasonHeroValue('topScorer', 10) → big 10`, unit `שערים`; `share = round(10/27*100) = 37`, so the unit line reads (182–184:) `37% מכל שערי המועדון . שערים . share` is computed only for `topScorer` and only when `totals.goals > 0`. Note the denominator is `totals.goals = 27`, which excludes the season's `guestGoals: 4` — the club actually scored 31.
- stat strip: 22 שחקנים 37 משחקונים, מחזורים (suppressed at 0), 7 שחקנים

`Cabinet` (`SeasonHistoryScreen.tsx:124–250`) draws nine fixed slots in `SEASON_TITLE_KEYS` order, three to a shelf, with an engraved socket where a title went unclaimed. Per slot: `SeasonMedal` (ring = tier, core = title tint, ribbon = title tint, `×N` badge when `streak > 1`), then the title name, then `names.slice(0,2).join(' · ')`, then `he.seasonTitleSharedWith(n-2)` when more, then `he.seasonTitleValue`, then `TIER_NAME[tier]` in words. The whole slot carries one `accessibilityLabel` and `SeasonMedal` is hidden from the accessibility tree (`SeasonMedal.tsx:51–54`).

The tiers s1 actually produces, with `completedRounds = 22` :

title	holder(s)	value	scale	tier
topScorer	הלן צברי	10	count [8,16,28]	כסף
topAssister	Nofar Tzabari	5	count [6,12,22]	ארד
mvp	all seven players	6	eveningScore [6.6,7.6,8.8]	ארד
topWinner	Lioz Madar	13	count [10,20,34]	כסף
mostLoyal	מתן לוי	19	19/22 = 0.864 vs [0.6,0.8,1]	זהב
cleanSheetKing	הלן צברי	13	count [5,11,20]	זהב
penaltyKing	הלן צברי	1.0	rate [0.7,0.85,1]	פלטינה
penaltyKeeper	הלן צברי	1.0	rate [0.7,0.85,1]	פלטינה
deadlyDuo	—	—	—	לא חולק

Two of these are worth the reviewer's attention. **The two platinum medals were won on two penalties each** (`penTaken: 2`, `penScored: 2` ; `penFaced: 3`, `penSaved: 3`) — `RATE_STEPS` has no volume floor, while `src/utils/penaltyStats.ts`, which decides the same-named crown on the club card, uses a Wilson lower bound written specifically to stop a one-shot 100%. **The headline title wears the weakest metal:** `eveningScore` is clamped to `[6,10]` and `6.0` is *also* the sentinel for "no mini-games recorded", so the seven-way `mvp` at exactly 6.0 is both structurally bronze and structurally shared. **ROUND4 #5** — `awardsDenominator` making `mostLoyal` permanently platinum — was resolved by deleting the field and the code that wrote it; `:159–170` records that both the finding and the first fix were wrong, and the divisor is now `season.completedRounds`. Ledger *shared-title-slot-names-nobody* is fixed: the names print and the suffix is additional.

The footer (`:281–289`) is `formatRange(startsAt, endsAt)` plus the `נסגרה ידנית / נתונים חלקיים` chips. **Neither chip renders on s1**, because the card carries neither field — and the ledger's *partialdata-only-on-the-sealnow-path* says s1 genuinely is partial (19 of 22 evenings predate the feature, assists only from 21.06, clean sheets from 17.08).

`formatRange ( :71–101 )` is where **ROUND4 #19** was fixed and a deeper problem was not. Both ends of s1 format to `2026 ספט`, so `from === to`, so the screen now falls through to `he.seasonRangeDays(formatDay(start), formatDay(end))` → `"16 בספט – 17 בספט"` printed directly under `22 מחזורים`. That is an improvement on the old `"2026 ספט – 2026 ספט"` and on the wrong first fix `"2026 ספט עד ספט"`, but it is still a 31.6-hour caption on a season holding twenty-two weekly evenings, because `startsAt` is the moment seasons were switched on. The dates are honest about the wrong event.

The 'void' variant renders `VoidRibbon ( :112–122 )` — one dashed grey line, `he.seasonVoidLine(no, day) = N עונה` . The 'noTitles' variant keeps the poster (hero `null` → the 35pt line is just `N עונה`) and replaces the cabinet with `he.seasonHistoryNoTitles` .

SeasonSummaryScreen **and** SeasonShareCard

src/screens/profile/SeasonSummaryScreen.tsx, model from src/services/seasonSummaryService.ts. Two sources, chosen by one rule (:425–446): if a seasonId was asked for, the archive is tried first — seasonSummary/{groupId}__{seasonId} — precisely so a player who has left the club and can no longer read /groups can still open the season they played. Only then does it fall back to the group document plus the live communityPlayerStats / communityPairStats rows.

completedRounds (:382, :512) is the one number this screen and the hall of fame must agree on. Closed → d.completedRounds from the archive (22). Running → seasons.playedRounds. Both were previously totals.rounds / communityStats.rounds, which is why the same season read 22 on one screen and 37 on another, and 0 משחקים for every timer-only club. **HEAD only**.

Render order (:403–699): hero (עונה N · club name, then either he.seasonsProgressRounds(played, target) for a running rounds season or formatRange, then העונה הסתיימה); the season picker chips from seasonChoices(seasons) when available.length > 1, each 44pt tall; then five cards gated on two independent conditions. me.hasData (seasonPersonal.ts:359) is rounds > 0 || games > 0 — attendance **or** mini-games, so a timer-only club no longer tells every player they played nothing. The champions card is gated on model.closed **only**, so a non-participant still gets אלופי העונה; the personal cards are gated on hasData.

The thirteen tiles (:519–534) read straight off PersonalSeason. Their denominators deliberately differ and the screen says so: goalsPerRound divides by rounds, assistsPerRound by asRounds, cleanSheetPct by csRounds, and partialCoverage (rounds > 0 && (csRounds < rounds || asRounds < rounds)) prints he.seasonPartialCoverageNote underneath. Four of the seven s1 rows have asRounds < rounds (e.g. הלין: rounds 27, asRounds 25, csRounds 17), so the note fires for them.

The standing card's three tiles use rankOf (seasonPersonal.ts:258–297), whose denominator is now rounds > 0 || games > 0 and whose rank is null unless **some** player has a non-zero value in that column — the fix for "three players got a gold medal for 1st place in goals decided by uid alphabetical order". Under them, he.seasonClubRounds(model.completedRounds).

SeasonShareCard (src/components/summary/SeasonShareCard.tsx) is the only artefact of this feature that leaves the app. Fixed 340pt, rendered off-screen at top: -10000 (:941–945), captured with captureRef, allowFontScaling={false} on **every** text. Tile set (:107–122): goals, assists and ערבי משחק **unconditionally**, then the six mini-game tiles only when me.rounds > 0 — 23 of the 93 production stat rows have evenings and no mini-games, and those players used to get a card of eight zeros. worthShowing (:95–99) now requires of > 1 **and** either top-3 in a club of ≥ 8 or the top third, closing both 3 מתוך 3 מקום and 30 מתוך 27 מקום. Titles print sharedWith (:209–211), so the seven holders of s1's כתר העונה no longer each send an identical exclusive-sounding claim. **All four of those are HEAD only**; 1.1.9 ships the mini-game-only tile set, bestRank.rank <= 3 with no club-size floor, and no sharedWith.

SeasonTitlesShelf — the profile

src/components/profile/SeasonTitlesShelf.tsx. Reads users/{uid}/seasonTitles — a collection written only by closeSeason — and sorts b.at - a.at || a.id.localeCompare(b.id) (:64), because every title from one season shares a millisecond. Each doc carries groupName **frozen at the moment it was**

won, so the row survives the club being renamed or the player leaving. Unknown `titleKey`s are skipped (:53). It renders **nothing at all** while loading and nothing when the player holds none (:87). For this club the collection holds 14 docs across 7 users, seven of which are `__s1__mvp` at value 6.

The Hebrew: מחזור, מחזור, and the strings the data can contradict

The feature's vocabulary lives in `src/i18n/he.ts`. The two units:

- **מחזור** = one whole evening. `he.seasonStatRoundsShort`: 'מחזורים', `he.communityStatsEvenings`: 'מחזורים', `he.seasonStatEvenings`: 'ערבי משחק'.
- **משחקון** = one mini-game inside it, recorded only in advanced mode. `he.seasonStatMiniShort`: 'משחקונים', `he.seasonStatRounds`: 'משחקונים', `he.communityStatsMiniGames`: 'משחקונים'.

`he.seasonsCardInfo` is the only string that defines the distinction for a user: "מחזור הוא ערב משחק שלם, לא משחקון" — "בולד בתוכו" — "a מחזור is a whole game-evening, not a single mini-game inside it." The 1.1.9 binary calls mini-games `משחקים` ("games") in at least three places the HEAD copy pass corrected: `communityStatsMiniGames` was 'משחקים', `seasonClubRounds` was ``${rounds}`` fed with `totals.rounds` (so "37 משחקים" for a season whose hall of fame said "37 משחקונים", and "0 משחקים" for every timer-only club), and `seasonPeerPartnerDetail` said `משחקים`.

The nine titles are defined once, in a `TITLE` constant **above** the `he` object (`he.ts:34–44`), because an object literal cannot read its own keys while it is being built — which is exactly how the `מלך → כתר` rename shipped half-done and produced `ROUND4 #26: הלן צברי` was `כתר השערים` on her hall-of-fame card and `מלך השערים` four times over on the club stats screen for the same season, one tap apart. At HEAD the club-stats leader labels point at `TITLE` (`he.ts:1356–1376`), so the two agree. **HEAD only**.

Strings whose claim the data can contradict, still true at HEAD:

- `he.seasonRangeDays` on `s1` → 16 בספט' 17 – בספט' over 22 מחזורים. The range is real; the event it measures is not the season.
- `he.seasonRangeUntil`: (to) => 'מתחילת המועדון עד' + to — "from the club's beginning until X". It is reserved for a season 1 with no `startsAt`, and the file now carries a ⚠️ doc-comment saying so. It reached the same-month case once and is one refactor away from doing it again.
- `he.seasonHistoryNoTitles` — "תואר ניתן רק למי שהגיע לפחות לחצי מערבי המשחק של העונה", "a title goes only to someone who attended at least half the season's evenings". The gate is `games >= ceil(D/2)` where `D = max(players.games)`, not the season length: on `s1` that was `ceil(19/2) = 10` of 22 evenings = 45%. The copy promises half; the code enforces half of whatever the best attendee managed.
- `he.seasonsToggleInfo` no longer promises nine titles (`s1` awarded eight; six of the nine need mini-games), but `he.seasonHistoryNoTitles` still blames attendance first for a season whose counter simply never advanced.
- `he.communityStatsScopeClosedInfo` — "אחוז ההתארגנות, הרצפים, הכימיה בין השחקנים ותארי המועדון נמדדים על המועדון" — "the organisation rate, the streaks, the chemistry and the club titles are measured over the club's whole life... and so they are shown only in the running season." At HEAD those three rows genuinely do read `lifetime`, so the tooltip is now true — but the `מחזורים` hero tile eight rows above it does not, and nothing on the card says which of the two scopes any given number is in.

- `he.seasonTitleNames.mvp` = `כתר העונה`, "the crown of the season", printed on seven people at once on the only season ever closed.
 - Gender: seven of the nine titles were `מלך` ("king"); the rename to `כתר` ("crown") is the only available fix because no gender field exists anywhere in the app. `he.seasonPeerVictimDetail` and `seasonPeerTormentorDetail` drop the pronoun entirely for the same reason, and `he.seasonHeroValue` returns `'הצלחה בפנדלים'` rather than `'מהפנדלים שבעט'`. `communityStatsMostLoyal` was the adjective `הכי מתמיד`; it now points at `TITLE.mostLoyal`.
 - `he.seasonPeersEmpty` — `"מי שיחק עם מי נספר רק במחזורים שמנוהלים במסך הלייב המתקדם"` — replaces an older `"זה יתמלא מעצמו"` ("this will fill itself in"), a promise a timer-only club can never see kept.
 - `he.seasonsConfirmSealedNow(0)` → `"0 מחזורים קיימים - תיסגר כעת 0"`: what this club's admin would read on the end-season dialog today, against a card that says the club has played 22.
-

12. Notifications, security rules, triggers and cost

Everything here is infrastructure the seasons feature borrowed rather than built, and most of the defects are at the seams — where a new notification type, a new collection and a new sweep were bolted onto plumbing that already had rules of its own. Both the deployed functions and the deployed ruleset are HEAD (front matter, *What is running where*); where an older deployment is what produced a document that is still in the database, that is said explicitly.

The announcement: `announceSeasonClosed`

`functions/src/index.ts:15324–15373` . One push per player, fanned out after the archive is sealed, never before.

```
// functions/src/index.ts:15332–15361
const snap = await db.collection('seasonSummary').doc(`${groupId}__${seasonId}`).get();
if (!snap.exists) return;
const players = (snap.data()?.players ?? {}) as Record<string, { rounds?: unknown; games?: unknown };
const ops: Promise<unknown>[] = [];
for (const [uid, row] of Object.entries(players)) {
  const rounds = typeof row?.rounds === 'number' ? row.rounds : 0;
  const evenings = typeof row?.games === 'number' ? row.games : 0;
  if (rounds <= 0 && evenings <= 0) continue;
  ops.push(createNotificationOnce({
    type: 'seasonSummary', recipientId: uid,
    entityType: 'group', entityId: groupId, reason: `season-${seasonId}`,
    payload: { groupId, seasonId, seasonNo, groupName },
  }));
}
```

Three deliberate choices, all defensible:

1. **The recipient list is the archive, not the club roster.** Somebody who played the first half of the season and then left the club is still told what they did; a member who never turned up is not sent a screen of zeros. The archive's `players` map is the only list that encodes "played this season".
2. **The filter is `rounds || games`, in that order.** `rounds` counts משחקונים (mini-games) and is written only by the advanced live screen. A club running the plain timer archives `rounds: 0` for every player of every season it will ever play, so filtering on `rounds` alone means a timer-only club closes its season and tells *nobody*. `games` is evenings attended and every club records it. The fix is in — in *this* function. It is not in the security rules (below), which is the whole of finding **season-archive-rule-gates-on-minigames**.
3. **Best-effort.** Wrapped in a `try`, failures counted and logged, never rethrown. A failed push must not roll back an archive that is already sealed and a live table that is already zeroed.

Point 3 has a hole: `createNotificationOnce` **never rejects** — every internal failure path returns `{ wrote: false, skipped: '...' }` — so the `Promise.allSettled / failed > 0` accounting at `index.ts:15363-15368` is dead code, and can only ever print `0/N`. That is **season-fanout-failure-accounting-dead**: a fan-out where every recipient silently failed and one where every recipient succeeded log identically.

Deterministic ids: `dedupeIdFor` and the cooldown buckets

`functions/src/notificationDedup.ts` is a hand-maintained mirror of `src/services/notificationDedup.ts` — the functions build has its own `tsconfig` `rootDir` and cannot import app source, so the two are kept in lockstep by hand and by a byte-comparison test.

```
// functions/src/notificationDedup.ts:105-121
export function dedupeKeyFor(input: DedupeInput): string {
  return `${input.type}:${input.recipientId}:${input.entityType}:${input.entityId}:${input.reason}`
}
export function dedupeIdFor(input: DedupeInput, nowMs: number = Date.now()): string {
  const cooldown = cooldownMsFor(input.type);
  const bucket = Math.floor(nowMs / cooldown);
  const raw = `${dedupeKeyFor(input)}__b${bucket}`;
  const safe = raw.replace(/[^A-Za-z0-9:_-]/g, '_');
  return safe.length > 480 ? safe.slice(0, 480) : safe;
}
```

The document id is the dedupe key plus a bucket number, and the write is `ref.create()` — atomic, failing with `AlreadyExists` (gRPC code 6) if the id is taken. That is the entire dedupe mechanism for `seasonSummary`: no unread-suppression query (`STRICT_UNREAD_DEDUP`, `index.ts:305`, holds only `gameCanceledOrUpdated`) and no aggregation (`AGGREGATE_ON_DUPLICATE`, `index.ts:352`, only `playerCancelled` and `gamePlayersJoined`). Its cooldown is **seven days** (`notificationDedup.ts:95`), and the bucket is a global epoch-week — `Math.floor(now / 604800000)` — so boundaries fall on Thursday 00:00 UTC.

Worked example, from production. The real close on `HhzIwmjMl1i5HS0Ght3p` (מועדון שכחת שושי) ran at `createdAtMs = 1789661534040 = 2026-09-17 16:12:14 UTC`. `1789661534040 / 604800000 → 2959`. The seven documents written are literally named:

```
seasonSummary:1IdtNEjbEXfiRSqvLrJVn99Nsfi2:group:HhzIwmjMl1i5HS0Ght3p:season-s1__b2959
```

Bucket 2959 opened 2026-09-17 00:00 UTC and closes 2026-09-24 00:00 UTC. Any re-close of `s1` inside that window — which is exactly what an undo-then-reclose is — mints the same seven ids, hits `AlreadyExists`, and pushes nobody (**reclose-after-undo-notifies-nobody**). The key itself is sound: `season-s1` sits in the `reason`, so a *different* season reaches everybody. It is the seven-day bucket that makes a same-season retry unreachable for up to a week.

Why nobody has ever received a season-summary push

This is the PO: `season-push-type-not-implemented-in-prod`.

`onNotificationCreated` (`index.ts:1874–1998`) is an `onDocumentCreated` trigger on `notifications/{id}` . Its shape:

```
// index.ts:1941–1951
const canonical = await canonicaliseNotificationPayload(notif.payload);
const message = buildMessage(notif.type, canonical);
if (!message) {
  await snap.ref.update({ delivered: true, deliveredAt: Date.now(), skipped: 'type-not-implemented' });
  return;
}
```

`buildMessage` (`index.ts:660–1122`) is a `switch` over the type with `default: return null` . Its `seasonSummary` case, `index.ts:1101–1112` :

```
case 'seasonSummary': {
  const no = typeof payload.seasonNo === 'number' ? payload.seasonNo : 0;
  const club = typeof payload.groupName === 'string' ? payload.groupName : '';
  return {
    title: no ? `עונה ${no} נגמרה` : 'נגמרה', // "Season {n} is over"
    body: club
      ? `מוכן – שערים, בישולים, ומי שיחק איתך הכי הרבה ${club} סיכום העונה שלך ב`
      : 'סיכום העונה שלך מוכן – שערים, בישולים, ומי שיחק איתך הכי הרבה',
  };
}
```

("Your season summary at {club} is ready — goals, assists, and who played with you the most.")

The case exists in source. It did not exist in the **deployed** reader on 17.09. The writer (`announceSeasonClosed`) had been redeployed twice that day; `onNotificationCreated` had not been redeployed since 07.09. Production proves it — all seven documents, read today:

field	value
type	seasonSummary
delivered	true
skipped	type-not-implemented
stats	(absent)
read	false
deliveredAt	1789661535069 – 1789661535113 (≈1.0 s after creation)

Seven documents, seven `type-not-implemented`. Six of the seven recipients had live FCM tokens at the time. The season-summary push — the thing the entire close builds towards — has been delivered to **zero people, ever**.

Two aggravations:

- **It is unreplayable (season-push-unreplayable)**. The docs are stamped `delivered: true`, so even if `onDocumentCreated` re-fired it would return at `index.ts:1879`. And `onDocumentCreated` does not re-fire on an update in any case — the same trap that hid 39 undelivered `teamsGenerated` documents for eleven weeks (ROUND4 #3). Recovery needs either a manual re-mint with a different id, or waiting out bucket 2959.
- **The bail happens *after* `canonicaliseNotificationPayload`**, which reads `groups/{groupId}` per notification (`index.ts:1848–1868`). The 17.09 close therefore spent seven reads of the same club document building a message it immediately threw away.

`gcloud functions describe puts onNotificationCreated` 's last update at **2026-09-18T18:16:54Z**, i.e. after commit `8235851` (18:14 UTC) — so the reader in production today does contain the case. Nothing has closed a season since, so this is still untested against a real delivery.

Delivery: `deliverBatch`, the dormant gate, and the toggle that does not exist

`deliverBatch` (`index.ts:1511–1789`) turns recipients into FCM tokens and sends. Three gates, in order, per recipient:

```
// index.ts:1548–1578 (abridged)
const prefKey = type === 'approved' || type === 'rejected' ? 'approvedRejected'
  : type === 'friendRequest' || type === 'friendRequestAccepted' ? 'friendRequest'
  : type;
for (const user of recipients) {
  if (user.notificationPrefs?.[prefKey] === false) { skippedPref++; continue; }
  if (dormantGated) {
    const ls = typeof user.lastSeenAt === 'number' ? user.lastSeenAt : 0;
    if (ls > 0 && ls < dormantCutoff) { skippedDormant++; continue; }
  }
  const userTokens = (user.fcmTokens || []).filter(t => typeof t === 'string' && t.length > 0);
  if (userTokens.length === 0) { skippedNoToken++; continue; }
  ...
}
```

The dormant gate. `DORMANT_PUSH_CUTOFF_MS` is 21 days; `DORMANT_SUPPRESSIBLE` (`index.ts:326–347`) lists the suppressible types. `seasonSummary` is deliberately **excluded**, with a comment arguing that the person a closing season is most for is exactly the one who played the first half and drifted off, and that it fires once per season. Sound in isolation; unsound in combination with the next paragraph.

The preference key does not exist. `prefKey` for `seasonSummary` falls through to the 1:1 branch, i.e. the key `seasonSummary`. But `defaultNotificationPrefs` (`src/types/index.ts:405–424`) has no such key, and `readNotificationPrefs` (`src/firebase/firestore.ts:453–463`) rebuilds the object by iterating the *default* key set:

```
const out: NotificationPrefs = { ...defaultNotificationPrefs };
(Object.keys(out) as (keyof NotificationPrefs)[]).forEach((k) => {
  if (typeof o[k] === 'boolean') out[k] = o[k] as boolean;
});
```

A stored `seasonSummary: false` is discarded on read and can never be written back, and the settings screen builds its switches from the same list, so there is no switch. Net effect: `seasonSummary` is the one type in the app that **ignores the dormant cutoff and has no off switch anywhere**. A comment at `index.ts:1539–1547` says exactly this. Same for `eveningSummary` (ROUND4 #20).

The stats block. After delivery, `index.ts:1990–1997` writes:

```
await snap.ref.update({
  delivered: true, deliveredAt: Date.now(),
  stats: { ok: totalOk, failed: totalFailed, skippedPref, skippedNoToken, skippedDormant },
});
```

`skippedDormant` was added only in the round-4 pass. Before it, 279 of the 1,206 notification documents carrying a stats block (23%) read `{ok:0, failed:0, skippedPref:0, skippedNoToken:0}` — a deliberately-suppressed announcement and a silently-broken dispatcher were indistinguishable (ROUND4 #14). There is no in-app notification inbox, so this block is the only durable record that a push existed; the corresponding Cloud Run log line ages out after 30 days.

What the seven recipients would have looked like had the reader worked. `/users/{uid}` and `/users/{uid}/private/push` are both denied to a client by the rules; these were read over the REST API with an owner access token, which bypasses rules, so a reviewer can reproduce the table with `gcloud auth print-access-token` and a `documents.get` on each path:

recipient	tokens	lastSeenAt	outcome
מתן לוי	4	2026-09-18	delivered
Lioz Madar	1	2026-09-16	delivered
Linoy Levi	1	2026-08-29	delivered
איציק לוי	1	2026-09-14	delivered
Nofar Tzabari	1	2026-09-04	delivered
Eliran Tzabari	1	2026-09-19	delivered
הלן צברי	0	2026-06-05	skippedNoToken

Note the last row: 104 days dormant, and *not* suppressed by the dormant gate (because `seasonSummary` is exempt) — she is skipped only because she happens to hold no token. None of the seven has a `notificationPrefs` map at all except `מתן לוי`, whose map has no `seasonSummary` key, because it cannot.

The deep link

`src/navigation/navigationRef.ts:359–373` :

```
case 'seasonSummary': {
  const seasonId = typeof data.seasonId === 'string' ? data.seasonId : undefined;
  if (groupId) {
    nav.navigate('ProfileTab', { screen: 'SeasonSummary', initial: false, params: { groupId, seasonId } });
    return true;
  }
  return false;
}
```

`SeasonSummary` is registered in all three stacks that can host it (`ProfileStack.tsx:162` , `CommunitiesStack.tsx:150` , `GameStack.tsx:172`), so the known nav-registration trap — `navigate()` silently no-ops for a screen missing from the active stack — does not apply. The push carries `groupId` and `seasonId` , so the route resolves. The screen then loads through `seasonSummaryService.load` (`src/services/seasonSummaryService.ts:411–540`), which is **archive-first** when a `seasonId` is given (`:433`) precisely so a departed player never touches `/groups/{id}` , which they can no longer read. That design is correct, and it is exactly what the rules then break.

Security rules

Two traps this repository has been bitten by, both of which are load-bearing here.

Trap 1 — a `list` query does not bind path wildcards. In `match /x/{docId}` , a query (as opposed to a single-document `get`) evaluates the rule with `docId` unbound — it reads as `null` . Any expression touching it raises a *Null value error* , and a raised error denies the **entire query** , not just one document. The workaround throughout this file is to answer `list` from a **field** on the document and to add a separate `allow get` for the by-id path.

Trap 2 — `.get(k, default)` defaults only on ABSENT, never on present-null. A document holding `k: null` returns `null` , not the default; the comparison downstream (`null > 0` , `uid in null`) then raises, and the request is denied. This bit production before, on `guestsOpenAt: null` blocking non-admin guest-add.

```
allow read: if isGroupMemberSafe(resource.data.get('groupId', ''));
allow get:  if isGroupMemberSafe(resource.data.get('groupId', '')) ||
  (request.auth != null &&
    request.auth.uid in resource.data.get('players', {})) &&
  (resource.data.players[request.auth.uid].get('games', 0) > 0 ||
    resource.data.players[request.auth.uid].get('rounds', 0) > 0));
allow write: if false;
```

The split is Trap 1 applied correctly, and keeping the participant clause *out* of `list` is subtler and also correct: a `list` is evaluated per document and the whole query dies if any one fails, so a participant clause there would deny a member's *entire* season history the moment it contained one season they sat out.

The defect: `games` in that clause was younger than the server that depended on it. HEAD has `games > 0 || rounds > 0`. The previous commit (f446474) has `rounds > 0` alone, and the `games` half landed in 8235851 at 2026-09-18 18:14Z. Both server sides — the close's participant filter and the push fan-out above — were corrected first. For the days in between, the server would push a season summary to every ex-member of a timer-only club and the deep link would land on `permission-denied`: the precise failure the clause exists to prevent. That is **season-archive-rule-gates-on-minigames**, confirmed independently three times in round 3 and again as ROUND4 #8.

It is now closed, and this is checkable. Release `cloud.firestore` points at ruleset 70cde240-0d57-4070-a311-9b1df55a1598, `updateTime 2026-09-18T18:14:42Z`; its 114,528-byte source `diff` is identical to `firestore.rules` at e35394a, participant clause included. (An earlier round left this open because the Firebase Rules API answered 403; the cause is a missing quota project, not a permission — add `-H "x-goog-user-project: soccer-app-52b6b"`.) The window was real and it has shut.

The clause has not fired in production yet, and the honest reason is that the only club that has ever closed a season is an *advanced-mode* club: all seven archived rows carry `rounds` between 24 and 26 alongside `games` between 18 and 19. The rules gate passes on `rounds` for every one of them. The trap is waiting for the first timer-only club.

Trap 2 is **latent, not live**, here. `closeSeason` (`functions/src/seasonRollover.ts:367-380`) writes every `PLAYER_SEASON_FIELDS` entry through `num()`, which coerces to a real number, so `games` and `rounds` are always present integers. `csRounds` / `asRounds` are deliberately *omitted* when absent (`seasonRollover.ts:376-378`) rather than written as `0` — which is the right shape for `.get(k, default)` and would still be safe if a rule ever read them. The exposure is one `?? null` away: a writer that ever emits `games: null` turns `allow get` into a hard deny for every participant, with no error the client can explain.

```
allow read: if isGroupMemberSafe(resource.data.get('groupId', ''));
allow get:  if isGroupMemberSafe(resource.data.get('groupId', ''));
allow write: if false;
```

`read` already means `get + list`, and both statements carry the **identical** expression, so `allow get` is a no-op. Harmless, but the comment above claims the pair exists for the list/get split — true of `/communityStats ( :2093–2094`, where the two expressions genuinely differ: field vs path wildcard) and false here.

Note also that the card is **members-only, with no participant clause**: a departed player can open the sealed archive of a season they played but cannot see the club's list of seasons, which makes the hall of fame a member-only surface by construction.

`/CLUBRECORDS/{GROUPID}` — `FIRESTORE.RULES:1813–1815`

```
match /clubRecords/{groupId} { allow read, write: if false; }
```

Fully server-only. This holds `eveningsSealed`, the all-time counter `seasons.roundsAtStart` is seeded from. Because the client cannot read it, season progress had to be mirrored onto the club document as `seasons.playedRounds ( index.ts:5219–5232 )` — a second copy of a number, and the origin of a good deal of the "22 on one screen, 0 on the next" family of defects. The rule is right; the mirror it forces is the cost.

`GROUPS.SEASONS` — `FIRESTORE.RULES:334–341 AND 376–396`

Create: `!( 'seasons' in request.resource.data )` — a club cannot be *born* holding a seasons block. Without it, anyone could plant `enabled: true` with an arbitrary `count`, and the hourly sweep would adopt the club without `enableClubSeasons` ever having run.

Update: the whole map is pinned.

```
( !affectedKeys().hasAny(['seasons']) ||  
  ( 'seasons' in resource.data &&  
    request.resource.data.seasons == resource.data.seasons ) )
```

Server-owned and immutable from the client, which is correct: four callables exist because enabling can seal a club's whole history and closing archives, zeroes every stat row and awards nine permanent titles. Note that this is a whole-map equality comparison, and the live cadence genuinely contains present-nulls (`{"targetRounds": 24, "months": null, "endsAt": null, "endsOn": null, "startsOn": null}` on the real club today) — map equality handles nulls fine, so Trap 2 does not apply.

`GAMES.SEASONID` — **NOWHERE**

`grep -n seasonId firestore.rules` returns **zero hits**. The stamp that decides which season an evening belongs to is not mentioned in the rules at all. Consequences (**games-seasonid-unpinned**):

- `allow create ( :794–822 )` validates a named handful of fields and says nothing about extras, so a member can create a game already carrying any `seasonId` they like.
- The organiser/admin branch of `allow update ( :1172–1244 )` can "mutate ANYTHING except terminal lifecycle states" plus `createdBy` and `groupId` — `seasonId` included.
- The server stamp is **write-once by absence-check** (`index.ts:5803–5812 : !(after as { seasonId?: string }).seasonId`), so a client-planted stamp is never corrected. It is accepted as authoritative by both `src/utils/seasonScope.ts` (client) and `functions/src/seasonCounters.ts` (server).

This is not hypothetical. Production, today, on the one club that runs seasons (`seasons = { currentNo: 2, currentId: 's2', count: 1 }` — so `s3` does not exist and never has):

date	game	status	seasonId
2026-06-28 → 2026-09-15 (19 games)	...	finished	(none)
2026-09-16	yA48XsyCEP	finished	s1
2026-09-17	nRHEgxVt3j	finished	s1
2026-09-17	TZ7IWEJWw	finished	s1
2026-09-17	DTNscolRoj	finished	s3

`DTNscolRoj` is stamped with a season the club has never reached, produced by a close/close/reopen sequence on 17.09. It belongs to no scope: the client's `inSeason()` rule (`seasonScope.ts:28–36`) says "stamped → must equal `currentId`", so it is excluded from `s2`; `s1`'s archive is sealed and cannot gain it; there is no `s3` card or archive to hold it. The stamp is write-once and finished games are client-read-only, so **nothing in the app can repair it (seasonid-stamp-orphan-unrepairable)**. It is also live: the seal's `tableZeroedAt / seasonRoundsAtStart split ( index.ts:5050–5106 )` exists *because* of this document — the code comment names it.

The nineteen unstamped games are not a bug in themselves; "your history becomes season 1" is what enabling the feature promises, and both sides implement it. But the client implemented it months before the server did, which is **unstamped-season-1-rule-disagrees-with-the-server-by-19**.

Triggers

`ONGAMEROSTERCHANGED` — `INDEX.TS:5295`

An `onDocumentWritten('games/{gameId}')` trigger and the largest function in the app, firing on **every** write to a game: join, cancel, arrival check-in, admin edit, status flip, mini-game commit. End to end it handles deletion bookkeeping (`gameDeletions` + the cancellation fan-out, `:5355–5428`), capacity notices, waitlist promotion and `spotOpened`, the pending-joiner buffer, holiday notices, the registration-open announcement, the join-request fan-out to admins — and four blocks that belong to seasons.

Block 1 — the club document, fetched once. `:5452–5476`.

```
let stampedSeasonId = '';
let groupSnapOnce: Promise<admin.firestore.DocumentSnapshot> | null = null;
const groupOnce = (): Promise<admin.firestore.DocumentSnapshot> => {
  if (groupSnapOnce) return groupSnapOnce;
  const p = db.collection('groups').doc(String(after.groupId ?? '')).get();
  groupSnapOnce = p;
  p.catch(() => { if (groupSnapOnce === p) groupSnapOnce = null; });
  return p;
};
```

Four consumers wanted `groups/{groupId}` and each opened it for itself, so one sealed evening read the same club document **three times** in a single invocation (**group-doc-read-three-times-per-sealed-evening**). Fixed. Note the deliberate detail: the *success* is memoised, never the failure — `??=` cached the rejected promise, so one transient blip cost the invocation its season stamp, its progress mirror *and* the join-request push, three blocks that used to fail independently (ROUND4 #22).

Block 2 — the season stamp. :5760–5851 .

```
const stampStatusChanged = before?.status !== after.status;
const stampRetryPending = (after as { seasonStampRetry?: boolean }).seasonStampRetry === true;
if (after.groupId && (stampStatusChanged || stampRetryPending) &&
    !(after as { seasonId?: string }).seasonId &&
    (after.status === 'active' || after.status === 'finished')) {
  const gSnap = await groupOnce();
  const seasons = gSnap.data()?.seasons;
  if (seasons?.enabled && seasons.currentId) {
    await event.data!.after.ref.update({ seasonId: seasons.currentId, ...clearRetry });
    stampedSeasonId = seasons.currentId;
  }
}
```

The status-transition gate is the fix for the second half of **group-doc-read-three-times-per-sealed-evening**: with the absence-check alone, a club that runs no seasons never gets a stamp written, so `!after.seasonId` stays true for ever and *every* write to an active or finished game re-read the club document to be told the same thing again. 194 of the 195 clubs in production have seasons off. The `seasonStampRetry` marker restores the retry the transition gate removed — an evening that loses the group read on both transitions would otherwise never be stamped, and an unstamped evening is filed under season 1 by both sides. No game in production currently carries the marker.

Block 3 — the seal's season mirror. :5050–5232 , inside `sealRoundSummary` . Reads `groupOnce()` , decides via the evening's stamp whether it credits the *running* season, and if so increments `groups/{id}.seasons.playedRounds` on the same batch that seals the evening.

Block 4 — close-on-target. :5269–5291 , after the batch commits.

```
if (seasonRoundsAtStart !== null && liveSeasons) {
  const playedNow = typeof liveSeasons.playedRounds === 'number' && liveSeasons.playedRounds >= 0
    ? liveSeasons.playedRounds + 1
    : eveningsSealed + 1 - seasonRoundsAtStart;
  const gName = (await args.groupOnce()).get('name');
  await closeSeasonIfRoundsTargetMet(groupId, gameId, { name: ..., seasons: liveSeasons, played: playedNow });
}
```

A rounds season closes the instant the evening that meets it is sealed, not at the top of the next hour. Only a rounds cadence can close this way; a date cadence is the sweep's job.

```
export const cronEvery60Min = onSchedule(
  { schedule: 'every 60 minutes', timeZone: 'Asia/Jerusalem', timeoutSeconds: 540, memory: '512MiB' },
  async () => {
    await runSweep('cleanupStaleGames', runCleanupStaleGames);
    await runSweep('sendPromotePrompts', runSendPromotePrompts);
    await runSweep('holidayGameNotices', runHolidayGameNotices);
    await runSweep('dailyCleanup', runDailyCleanupIfDue);
    await runSweep('clubActivity', runClubActivityIfDue);
    await runSweep('seasonRollovers', runSeasonRollovers); // ← LAST
  },
);
```

Six sweeps, **sequential**, one 540-second budget. The seasons sweep runs last and inherits whatever the other five leave; `runSweep` swallows each sweep's exception so one failure cannot starve the rest, but it cannot give back time. Two structural notes: `'every 60 minutes'` is an *interval*, not a cron expression, so `timeZone` is inert (P3 `cron-is-an-interval-not-a-clock`); and the timeout was raised from the 60-second default specifically because a close is a transaction per player plus a transaction per pair plus a batch of titles.

`runSeasonRollovers` (:16055–16283) defends its position with two constants: `SEASON_SCAN_PAGE = 200` (:16026) and `MAX_CLOSES_PER_SWEEP = 5` (:16042). Without the second, one sixty-player club with a few thousand pair documents can spend the whole remainder and every club after it in the scan is silently never reached.

The scan itself (**sweep-scan-fetches-the-whole-group-document**, fixed at HEAD):

```
let q = db.collection('groups')
  .where('seasons.enabled', '==', true)
  .select('name', 'seasons')
  .orderBy(admin.firestore.FieldPath.documentId())
  .limit(SEASON_SCAN_PAGE);
if (cursor) q = q.startAfter(cursor);
```

This reduces bytes, not billed reads. Firestore charges one document read per document returned regardless of projection; `.select()` trims the payload (2,259 bytes → 666 measured) and `.limit()` + cursor bound memory, but the read count for a club with seasons enabled is unchanged at one per sweep pass. The cost table below counts reads, not bytes, and is not improved by this fix.

due-but-blocked-is-invisible-forever, also fixed at HEAD: a due season that `clubIsQuiet` refuses used to emit one `console.log` per hour, for ever, while a season that *threw* went into the `errors` inbox a person reads. The fix stamps `seasons.dueBlockedSince` the first hour a due season cannot close and, past `DUE_BLOCKED_STUCK_MS` (24 h), calls `reportServerError({operation: 'seasonRolloverBlocked'})` — reported, never thrown, so the sweep still reaches every other club. Verified today: the production `errors`

collection holds **21 documents**, none of them seasons-related and none with `platform: 'server'` , which is consistent with the old behaviour and means this path has never yet produced a row. The club's live `seasons` map carries no `dueBlockedSince` .

Cost

Counts are Firestore **document reads**. The branch this work sits on is named `perf/firestore-read-costs` , so this is the ledger it exists for.

Today's scale, read live on 2026-09-19: **195 groups**, 1 with `seasons.enabled == true` , 95 games (91 finished), 93 `communityPlayerStats` rows, 717 `communityPairStats` rows, 675 users, 1,231 notifications.

PER SEALED EVENING

club state	reads attributable to seasons
seasons OFF (194 clubs)	2 — one <code>groups/{id}</code> on the <code>active</code> transition, one on <code>finished</code> , both via <code>groupOnce</code> and both answering "no". Before the transition gate this was one read per <code>write</code> to the game while active or finished: dozens per evening, for ever.
seasons ON, target not met	1 — <code>groupOnce</code> , now shared by the stamp, the seal mirror and the close check. Was 3.
seasons ON, target met	1 (<code>groupOnce</code>) + <code>clubIsQuiet</code> : 2 queries ($\leq 2 + \leq 3$ documents) and up to 3 <code>roundSummaries</code> gets, minus the just-sealed evening. Then a full close.

Writes per sealed evening with seasons on: one `seasonId` stamp (once in the evening's life) and one `seasons.playedRounds` increment folded into the existing seal batch.

PER CLOSE

Measured against the real club and then extrapolated. `closeSeason`
(`functions/src/seasonRollover.ts:306-...`) opens with three parallel reads:

```
const [psSnap, csSnap, pairSnap] = await Promise.all([
  db.collection('communityPlayerStats').where('groupId', '==', groupId).get(),
  db.collection('communityStats').doc(groupId).get(),
  db.collection('communityPairStats').where('groupId', '==', groupId).get(),
]);
```

step	reads, real club (7 players)	reads, 60-player club
<code>performSeasonClose</code> pre-flight <code>groups/{id}</code>	1	1
<code>communityPlayerStats</code> where <code>groupId</code>	7	60
<code>communityStats/{id}</code>	1	1
<code>communityPairStats</code> where <code>groupId</code>	314	~2,700
<code>getAll</code> of <code>/users</code> for display names	7	60

step	reads, real club (7 players)	reads, 60-player club
seasonSummary latch get	1	1
per-player wind-back transactions (1 read each)	7	60
communityStats wind-back transaction	1	1
sealedEveningsOf → clubRecords/{id}	1	1
lifecycle advance transaction	1	1
announceSeasonClosed → seasonSummary re-read	1	1
onNotificationCreated ×N: canonicalise groups/{id}	7	60
onNotificationCreated ×N: loadUsers (root + private/push)	14	120
total	≈363	≈3,067

The pair row dominates and is almost pure waste (**guest-pairs-make-every-close-read-15x-what-it-archives**). Of the club's 314 pair documents, 293 are guest pairs — a guest id is minted fresh for every game, so the collection gains a permanent document per stranger per night and nothing ever deleted one. The archive keeps none of them: 314 documents read, at most 21 archivable, and on the 17.09 close every one of those 21 was already empty. Guest pairs are now deleted during the wind-back (seasonRollover.ts:944–948 , deployed); all 314 are still on disk because no close has run since that landed.

Two more line items are pure overhead: the seven canonicalise reads of one club document (one per notification, all identical, all discarded when buildMessage returned null), and the /users read per close that exists only to stamp closedBy / closedByName — two fields **read by nothing (closedby-write-only)**.

PER SWEEP RUN

runSeasonRollovers costs, per hour:

- **1 read per club with** seasons.enabled == true — the paged scan. Today: 1.
- Clubs not yet due stop there: a date cadence is answered from the document in hand, and completedRoundsOf is called only for a rounds cadence, and then only via the playedOnce() memo (:16114–16124) so the value is computed once instead of the two identical derivations it used to do.
- A due club adds clubIsQuiet : 2 game queries plus up to 3 roundSummaries gets ≈ 5–8.
- A due-and-quiet club adds a full close (table above).

scale	sweep reads/hour	sweep reads/day
today (1 enabled club, not due)	1	24
1,000 enabled clubs, none due	1,000	24,000
1,000 enabled clubs, 5 due and quiet	1,000 + 5×~363 ≈ 2,815	~67,600

24,000 reads/day is the floor at 1,000 clubs, it buys nothing on the ~999 not due, and it is the number `.select()` does *not* reduce. The obvious improvement — a `where('seasons.cadence.endsAt', '<=', now)` filter — is not implemented, and cannot cover a rounds cadence, whose due-ness is not a field on the club document.

The 5-close ceiling interacts badly with scale: at 1,000 clubs on a weekly rhythm, more than five will routinely be due in the same hour, and a club deferred behind a busier neighbour has no priority or fairness mechanism — the scan order is document-id ascending, for ever. Nothing tests any of this (**nothing-tests-the-sweep**: zero tests for `runSeasonRollovers`, `clubIsQuiet`, or the ordering).

PER STATISTICS-SCREEN OPEN

Season-scoped club statistics come from `gameService.getCommunityStats` (`src/services/gameService.ts:949–1124`), whose single query is:

```
const q = query(col.games(),
  where('groupId', '=', groupId),
  where('status', 'in', ['finished', 'cancelled']),
  orderBy('startsAt', 'desc'),
  limit(200));
```

Up to 200 game documents per open, season-scoped or not — the season filter is applied in memory afterwards, via `inSeason()`. On the real club that is 23 reads; on a club with three years of history it is the full 200, every time the screen is opened, with no cache. Plus, on the same screen: the club document, `communityStats/{groupId}`, and a `communityPlayerStats where groupId` scan (7 today, 60 on a large club).

The hall of fame is cheap by design: `seasonHistoryService.list` (`src/services/seasonHistoryService.ts:116–145`) reads `seasonCards`, not archives — 2,473 bytes per card against 20,596 bytes for the same season's archive, with the winners already resolved. One read per finished season.

The personal season summary (`seasonSummaryService.load`) is two-shaped: a **closed** season is one read of `seasonSummary/{groupId}__{seasonId}` plus a peer-name resolve; the **running** season is `groups/{id}` + `communityPlayerStats where groupId` + two `communityPairStats` queries (`a == me`, `b == me`), so it scales with roster size rather than with the season.

Finally, the rules themselves bill reads. `isGroupMemberSafe` (`firestore.rules:101–106`) performs an `exists()` **and** a `get()` on `groups/{gid}` — two document accesses per evaluation, cached within a single request. So every `seasonCards` list, every `seasonSummary` get and every `communityStats` read adds two billed reads on top of the documents returned. On the archive's `allow get` the participant clause is free by comparison: membership is a key in the document being read, so it costs nothing extra.

13. What is tested, what is not, and the open questions

How the suites are run, and by whom

There are two independent test surfaces and no automation connecting them to anything.

The unit suite is Jest. `jest.config.js` is five lines:

```
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/tests'],
  moduleNameMapper: { '^@/(.*)$': '<rootDir>/src/$1' },
  transform: { '^\\.\\.\\.tsx?$': ['ts-jest', { isolatedModules: true }] },
};
```

`roots` is `tests/` only, and the transform matches `.ts / .tsx`. The rules suite under `tests/rules/` is written in `.mjs` against Node's built-in `node:test` runner, so `npx jest` does not see a single one of its files. Running `npm test` at the repo root reports a green run while 146 security assertions have not been executed.

The rules suite is its own npm project (`tests/rules/package.json`) with its own `node_modules` :

```
"test": "node --test --test-concurrency=1 --test-reporter=spec *.test.mjs"
```

`--test-concurrency=1` is load-bearing. Five of the eight suites (`firestore` , `antiHijack` , `clubIsolation` , `oldClientCompat` , `roundHistoryAccess`) share `projectId: 'demo-soccer'` and each calls `clearFirestore()` in its setup; run in parallel they wipe each other's fixtures. `seasonSummary.test.mjs` is one of the three that took its own project id (`rules-season-summary`), so the seasons rules tests are at least isolated from that hazard.

There is **no CI**. There is no `.github/` , no workflow file, no `.husky/` , and no non-sample hook in `.git/hooks` . Nothing runs Jest, the rules suite, or `tsc` unless a person types the command. `npm run typecheck` (`tsc --noEmit`) is clean at HEAD for both the app and `functions/` , but Metro — the React Native dev server — does not typecheck, which is how three seasons defects of exactly the class a compiler catches reached the production `errors` collection on 18.09 from the developer's own emulator:

fingerprint	message	surface
toxitz	<code>_he.he.seasonsCardOfTarget</code> is not a function (it is undefined)	<code>SeasonsCard</code>
1ytrra f	Cannot read property 'totalFinished' of undefined	<code>CommunityStatsScreen</code>
3lov71	Property 'winsPlaceIsFactual' doesn't exist	<code>assistantInsightsService</code>

All three are `appVersion 1.1.5` with a `10.0.2.2:8081` Metro bundle URL, so they are dev-emulator reports rather than store users — but they are the only seasons entries the error inbox has ever held, and they are all "a symbol that did not exist yet". The production `errors` collection holds 21 documents in total and **not one of them comes from the seasons backend**, which is a fact about the backend's error reporting rather than about its correctness: the hourly sweep's failure path is `console.log`, and only a `throw` reaches the inbox.

The seasons unit suite, file by file

Twenty-three Jest files touch seasons; together they are 362 assertions and they all pass today (2.7s). Every one of them was written between 13.09 and 18.09.2026 — that is, *after* the feature shipped in 1.1.7 on 16.09 and mostly *during* the audit. Season 1 of the only club that runs seasons was closed by this code at `2026-09-17T16:12:06Z`, at which point the close had no functional test of any kind.

File	n	What it pins	What it leaves free
<code>seasonCounterReconciliation.test.ts</code>	38	All four answers to "how many <code>מחזורים</code> " agree on the real club: the client games scan, <code>inSeason</code> , the counter subtraction, and <code>countSeasonParticipants</code>	One club's fixture. A second club shape is not exercised
<code>seasonMedalTier.test.ts</code>	40	The 60/80/100% steps, the 6–10 MVP range, all nine titles gradable, <code>titleStreak</code>	Nothing renders a medal in a test
<code>seasonAwards.test.ts</code>	32	Eligibility <code>ceil(D/2)</code> , penalty floors, ties shared, the MVP scale floor, the nine-key set	The choice of <code>D</code> — the denominator is an argument here
<code>seasonPersonal.test.ts</code>	29	<code>buildPersonalSeason</code> : rates, ranks, peer orientation, guest exclusion, three distinct rival rows	<code>seasonSummaryService</code> , which chooses the inputs
<code>seasonCardVariant.test.ts</code>	22	full / card / ribbon / void classification; a timer-only club is never void; <code>heroWinner</code> , <code>heroSeasonId</code>	The screen that renders the variant
<code>seasonLifecycle.test.ts</code>	22	<code>isSeasonDue</code> , the quiet seam, rounds remaining, mid-season retarget validation, month clamping	The client copy only; the server has its own
<code>seasonActivation.test.ts</code>	21	<code>planActivation</code> for every cadence × choice × history combination, including the pre-picker client	The settings screen that builds the input
<code>seasonDates.test.ts</code>	21	Month-end clamping, leap years, club-midnight vs UTC, three years of arithmetic	—
<code>seasonCloseNumbers.test.ts</code>	20	<code>windBackRow</code> , <code>restoreRow</code> , <code>seasonPairKey</code> , <code>awardsDenominatorOf</code> , <code>sealedCardEvenings</code>	Which documents <code>closeSeason</code> reads, in what order, in which batch
<code>eveningPlayedMirror.test.ts</code>	17	Five client/server mirror pairs: byte identity and differential execution	The deployed bundle
<code>seasonSweep.test.ts</code>	12	<code>seasonFinishLine</code> and <code>isSeasonDue</code> from <code>functions/src/seasonCounters.ts</code>	The sweep itself, <code>clubIsQuiet</code> , <code>performSeasonClose</code>

File	n	What it pins	What it leaves free
<code>seasonOpensWithOffset.test.ts</code>	11	All five season-opening object literals stamp <code>roundsAtStart</code> and <code>playedRounds</code>	Whether the values are right
<code>groupSeasonsReader.test.ts</code>	10	The client deserializer keeps every field the server writes, including <code>targetHistory</code>	—
<code>groupHydrate.test.ts</code>	10	A failed club refresh does not empty the store	—
<code>seasonRolloverFields.test.ts</code>	9	<code>__seasonFields</code> — the exact list of counters a close zeroes, and their pairings	That the list is applied
<code>seasonArchiveTable.test.ts</code>	8	<code>parseSeasonTable</code> over a hand-copied archive fixture	Key-set parity with the writer
<code>seasonChoices.test.ts</code>	7	<code>lastClosedSeason</code> , the season picker order	—
<code>seasonScopedStats.test.ts</code>	7	<code>inSeason</code> — including "unstamped means season 1" — over the real club's documents	—
<code>seasonAwardsMirror.test.ts</code>	6	The title rules agree client/server at denominators 0, 1, 2, 13, 19, 20, 22, 40	—
<code>seasonCadenceClean.test.ts</code>	5	Every cadence literal nulls the other kind's fields and declares its <code>type</code>	—
<code>sealedCounterAtomicity.test.ts</code>	5	Neither evening counter is ever written as <code>x + 1</code>	The race itself
<code>seasonCardParticipants.test.ts</code>	5	<code>countSeasonParticipants</code> on a timer-only club	—
<code>seasonSeedHistory.test.ts</code>	5	<code>seasonSeed</code> starts a season at zero regardless of club history	—
<code>allTimeTable.test.ts</code>	9	Live season + archives merge; coverage denominators are dropped, not invented, when one side never measured	—

The fixture that most of the newer files share, `tests/fixtures/realClub.ts`, is a hand-copied snapshot of the one club that runs seasons, taken on 18.09.2026: 22 evenings of which 19 carry no `seasonId`, one `unverified` evening, one cancelled, `eveningsSealed: 10`, `roundsAtStart: 7`, a sealed card of `completedRounds: 22` beside `totals.rounds: 37`. Its own header explains why it exists — every fixture before it described a club that does not exist. Nothing re-verifies it against production; it will drift silently.

Measured coverage of the server modules

File	% Stmts	% Branch	% Funcs	Uncovered Line #s
functions/src	45.14	33.99	72.41	
seasonActivation.ts	97.22	100	100	179
seasonAwards.ts	92.45	83.33	87.09	249–250
seasonCounters.ts	97.5	97.14	100	75
seasonDates.ts	85.24	69.23	88.88	74–76, 128–135
seasonParticipants.ts	100	100	100	
seasonRollover.ts	12.62	5.50	28.00	308–1026, 1072–1252
seasonSeed.ts	100	100	100	

`seasonRollover.ts` is 1,253 lines. The uncovered ranges are exactly `closeSeason` (declared at line 307) and `reopenSeason` (line 1067). Everything the file has a test for is the arithmetic helpers above them — `windBackRow` (62), `restoreRow` (84), `seasonPairKey` (104), `awardsDenominatorOf` (133), `sealedCardEvenings` (155) and the `__seasonFields` list (1034). The two functions that actually mutate a club's history run 5.5% branch-covered.

`functions/src/index.ts` is 17,081 lines and is **never imported by any test**. The seasons lifecycle occupies roughly lines 15,405–17,081 of it, plus the stamp block at 5,802–5,850, the push fan-out at 15,333–15,367 and the notification body at 1,101. All of it is reached only as text.

The MIRROR tests, and what they cannot see

Cloud Functions cannot import from the app's `src/`, so six modules exist twice. Each server copy carries a marker line and is required to be byte-identical below it:

```
const MARKER =
  '/* ---- everything below this line is a copy of the client file ----';
const copied = server
  .slice(server.indexOf(MARKER) + MARKER.length)
  .replace(/^\\n+/, '')
  // Cloud Functions have no `@/` alias, so that one line is allowed to
  // differ – and ONLY that one.
  .replace(/from '\\./seasonDates'/g, "from '@utils/seasonDates'");
expect(copied).toBe(client);
```

`eveningPlayedMirror.test.ts:32` declares five pairs — `eveningPlayed`, `seasonParticipants`, `seasonSeed`, `seasonDates`, `seasonActivation` — and `seasonAwardsMirror.test.ts:23` covers the sixth, `seasonAwards`.

Two defects in this mechanism were found by the audit and both are fixed at HEAD, but the second is the one a reviewer should understand:

1. `mirror-each-ignores-its-own-params` (**LEDGER**). The `describe.each` table declared five module pairs and the body hard-coded `src/utils/eveningPlayed.ts`. Four of the five mirrors were never compared; the suite reported ten green assertions that were two assertions run five times. The comment at lines 47–55 now records this.
2. **Byte identity is not evidence**. Two identical copies of a *wrong* rule pass a diff. Until 18.09 nothing in either file ever *executed* the server copy, so the server half of six rules had no test at all — only a diff. Both files now also load both modules and run them side by side: 480 evening shapes through `eveningPlayStateWithReason`, every club-document shape through `seasonSeed`, three years of month ends through `seasonDates`, every settings-screen submission through `planActivation`, and the real club's seven rows through `computeSeasonAwards` at eight different denominators.

The limitation that remains is structural and the reviewer should not overlook it. **The mirror tests read** `functions/src/`. **Production runs** `functions/lib/`, the `tsc` output, which is gitignored (`.gitignore:58`). Nothing in the repo compares the deployed bundle against either copy. This is not hypothetical: ROUND4 #1 established that both 1.1.9 store binaries were built from commit `4de80d0` while the backend they talk to was HEAD — twenty client files and +2,636 lines apart — and the audit's only method for checking what was actually deployed was downloading the function bundles and grepping them. A green mirror run proves `src/utils/x.ts` and `functions/src/x.ts` agree in the working tree. It says nothing about what the user's phone and the running function are doing.

A third limitation: agreement is not correctness. `seasonAwardsMirror.test.ts:65` runs the real club's sealed season through both copies and asserts they produce the same answer. They do. The answer they agree on gave שחקן העונה ("player of the season") to all seven members of the club at a value of 6.0, which is the evening-score sentinel for "no mini-games were recorded". Both halves were wrong in perfect agreement.

The SOURCE-GREP guards

Three rules cannot be unit-tested from a Node process — a Firestore merge into a nested map, a read-modify-write race, and a field missing from one of five inline object literals. The repo guards them by reading the backend as a *string* and asserting its shape. The three guards are `seasonCadenceClean.test.ts`, `seasonOpensWithOffset.test.ts` and `sealedCounterAtomicity.test.ts`.

As shipped, all three read exactly one file — `functions/src/index.ts` — with hand-written regular expressions that terminated on indentation. The cadence guard's original matcher was:

```
/(?:cadence|next) = \{[\s\S]*?\n {4,6}\};/
```

Two failure modes follow directly. **Indentation-sensitive**: the `\n {4,6}\};` terminator means a literal nested one level deeper, or written on a single line, is simply not matched. **Single-file**: a writer moved into `seasonRollover.ts` — or into a file created tomorrow — is invisible. Both are the same flaw: *the guard describes where the code is today rather than what it must not do*. And a guard that matches nothing asserts nothing while reporting green: the cadence file reports four passing tests over an empty array just as happily as over four literals.

`tests/fixtures/serverSource.ts` (added 18.09) replaces both properties. `serverFiles()` walks all of `functions/src/`, skipping `node_modules` and `lib`, and returns each file's text with a repo-relative path so a failure names something greppable. `objectLiterals(marker)` finds the opening brace after the marker and then **counts braces** to the close, so indentation is not part of the rule:

```
const open = file.text.indexOf('{', m.index + m[0].length - 1);
let depth = 0, i = open;
for (; i < file.text.length; i += 1) {
  if (file.text[i] === '{') depth += 1;
  else if (file.text[i] === '}') { depth -= 1; if (depth === 0) break; }
}
```

`codeLines()` strips comments so a guard looking for a forbidden write does not fire on the comment explaining why it is forbidden — and, more importantly, is not *satisfied* by one.

What is still fragile, and a reviewer should treat these as weaker than the tests around them:

- **The guards are count assertions.** `expect(cadenceLiterals.map(c => c.where)).toHaveLength(4)` and `expect(blocks.map(b => b.where)).toHaveLength(5)`. A sixth season-opener fails the count, which is the intent — but it fails on arithmetic, not on behaviour, and the fix ("update the number") is one character away from the fix that defeats the guard.
- **They match identifiers and string literals.** `/type: 'rounds'/`, `/currentId:/`, `/playedRounds:\s*eveningsSealed/`. Renaming a field, or building the literal by spread from a helper, quietly removes code from the guard's view. `seasonOpensWithOffset.test.ts` already has to special-case `...seasonSeed()` for this reason.
- `sealedCounterAtomicity.test.ts:63` **is the only guard that proves it can see its own subject** ("and the guard is reading the file the seal is actually in"). The other two do not. That assertion is the pattern the other guards need.
- They read `functions/src/`, with the deployment gap described above.

The honest inventory of untested code

Path	Where	Functional test
<code>runSeasonRollovers</code> — the hourly sweep	<code>index.ts:1</code> <code>6055</code>	none
<code>clubIsQuiet</code> — the close permission gate	<code>index.ts:1</code> <code>5466</code>	none
<code>performSeasonClose</code> — archive + titles + push + lifecycle	<code>index.ts:1</code> <code>5790</code>	none
<code>closeSeasonIfRoundsTargetMet</code> — the on-seal close	<code>index.ts:1</code> <code>5943</code>	none

Path	Where	Functional test
<code>closeSeason</code> — the wipe and the archive	<code>seasonRoll over.ts:307</code>	none (helpers only)
<code>reopenSeason</code> — the undo	<code>seasonRoll over.ts:1067</code>	none (helpers only)
The <code>seasonId</code> stamp trigger + <code>seasonStampRetry</code> marker	<code>index.ts:5802</code>	none
<code>playedEveningsOfSeason</code> / <code>completedRoundsOf</code>	<code>index.ts:15599 , 15695</code>	none
The six callables (<code>enableClubSeasons</code> 16286, <code>disableClubSeasons</code> 16613, <code>updateSeasonTarget</code> 16627, <code>reopenLastSeason</code> 16848, <code>endSeasonNow</code> 16971)	<code>index.ts</code>	none
<code>onNotificationCreated</code> and the <code>seasonSummary</code> push body	<code>index.ts:1874 , 1101</code>	none
The push fan-out	<code>index.ts:15333–15367</code>	none
<code>seasonSummaryService.ts</code> (636 lines), <code>seasonService.ts</code> (274)	<code>src/services/</code>	none
<code>SeasonsSettings.tsx</code> (1,150), <code>SeasonSummaryScreen.tsx</code> (951), <code>SeasonHistoryScreen.tsx</code> (699), <code>SeasonPoster</code> , <code>SeasonShareCard</code> , <code>SeasonsCard</code> , <code>SeasonMedal</code> , <code>SeasonConfirmSheet</code> , <code>SeasonTitlesShelf</code>	<code>src/screens</code> , <code>src/components</code>	none — there is no component-render test in this repo at all

Stated plainly, because it is the single most important sentence in this section: **the only unattended path in the app that destroys production data has no functional test**. The phrase is the codebase's own, from the comment on the cron that carries it (`index.ts:13589`):

```

export const cronEvery60Min = onSchedule(
  {
    schedule: 'every 60 minutes',
    timeZone: 'Asia/Jerusalem',
    // The default is 60 seconds, and this job now contains the season sweep –
    // the only unattended path in the app that destroys production data.
    timeoutSeconds: 540,
    memory: '512MiB',
  },
  async () => {
    await runSweep('cleanupStaleGames', runCleanupStaleGames);
    /* ... four more ... */
    await runSweep('seasonRollovers', runSeasonRollovers);
  },
);

```

Every sixty minutes, unattended, that function may zero a club's `communityPlayerStats`, `communityStats` and `communityPairStats`, write a `seasonSummary` with `create()` (which is one-shot), write nine `seasonTitles` onto players' profiles, and push every participant. `clubIsQuiet` is the only thing standing between a club and that happening mid-evening, and the fix that unblocked the clubs a legacy evening had frozen out — the `endedBy !== 'admin' && endedBy !== 'auto'` skip for legacy closes at `index.ts:15557`, discussed with its evidence and the limits of that evidence in §5 — landed with no test exercising it. The resume path that exists because a close can be killed halfway is likewise untested; so is the archived-latch that is supposed to stop a concurrent seal-close and sweep from resetting the *new* season (`LEDGER perform-close-writes-next-season-even-when-nothing-was-archived`).

Two more absences worth naming. Nothing tests the Hebrew vocabulary: the latest commit at HEAD (`e35394a`, "The last six labels that still called a משחקון a משחק") was, in its own words, "caught on the device, not in the diff" — the club stats screen printed "37 משחקים" (37 *games*) beside "22 מחזורים" (22 *evenings*) while the hall of fame one tap away called the same 37 "משחקונים" (*mini-games*). `copyDirectionality.test.ts` bans horizontal arrows in user-facing strings and `format.test.ts` pins the Hebrew plural forms of the season countdown; neither can catch a unit word. And nothing reconciles a *sealed* archive against the current rules: the s1 card still names seven MVPs at 6.0 because it was written before the scale floor existed, and `seasonAwards.test.ts:327` now asserts that cannot happen again — for future seasons only.

The rules test suite

```

brew install openjdk@21 # firebase-tools requires JDK 21+
export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"
firebase emulators:start --only firestore --project demo-soccer
cd tests/rules && npm install && npm test

```

One practical trap the README does not mention: `firebase.json` declares no `emulators` block, so the CLI also starts the Emulator UI on port 4000 and **aborts the whole run** if that port is taken (`Error: Could not start Emulator UI, port taken`). Adding `"emulators": { "ui": { "enabled": false } }` locally is the workaround.

The ruleset this suite is run against is also the one in production: release `cloud.firestore` points at a ruleset whose source is byte-identical to `firestore.rules` at `e35394a` (§12). So a green run here is a statement about the deployed rules, which is unusual in this repository and worth noting — the Jest suites make no such claim about the deployed functions.

Run against HEAD on 19.09.2026 the suite reports:

```
i tests 146    i suites 14    i pass 142    i fail 4      i duration_ms 7476
```

The four failures, and what each actually is:

1. `games: self can join an open community game` (`firestore.test.mjs:179`) — expects success, gets `PERMISSION_DENIED: Unable to evaluate the expression as the maximum of 1000 expressions to evaluate has been reached. for 'update' @ L824`. Not a rules bug in the ordinary sense: the `/games allow update` at `firestore.rules:824` is one long OR chain and Firestore stops evaluating a rule after 1,000 expressions. The rules file documents this at line 826 — the publish-split branch was moved to the front *because* "on a real game document (82 fields) the branches below burn the whole budget and the request is denied before this one is ever reached. That is what actually failed in production."
2. `OLD-CLIENT manual-offer cancel` (`oldClientCompat.test.mjs:61`) — the same ceiling, same rule, same line.
3. `groupsPublic: admin of canonical group can create the public mirror` — a stale test. It writes `memberCount: 2`; `firestore.rules:627` caps a create at `<= 1`. The test predates the cap.
4. `notifications: cannot fake an "approved" push to inflate a join` — a test that contradicts its own comment. It `assertFails` on a `spotOpened` create, then explains three lines later that "spotOpened IS in the client whitelist — that's intentional". Stale, not a hole.

So two of the four are test rot and two are a **live production constraint that the seasons rules sit directly downstream of**. The expression ceiling is not confined to the failures. Over the whole run, forty requests logged `maximum of 1000 expressions`: twenty-two of them against `@ L343` (the `/groups allow update`) and eighteen against `@ L824`. Every one of the seven tests that prove `groups.seasons` cannot be touched from a client (not an admin adding it, not an admin rewinding one that exists, and an ordinary member certainly does not, not deleting the field, not one nested number, not riding along on a self-leave, not riding along on a join request) **passes with a budget-exhaustion error in the log**. The deny is real, but it is not proof the rule denied — it is proof the engine gave up. If the `seasons` immutability clause were removed tomorrow, those seven tests would very likely still pass. That is the most serious weakness in the rules coverage of this feature and it has not previously been reported.

What the seasons rules suite does pin, and pins well — 40 tests in 12 groups, all passing, with its own project id:

- A club member reads a `seasonSummary` by id and by list, answered from the `groupId` field, never the path wildcard (the `list -does-not-bind` trap that once made `communityStats` unreadable).

- A **departed participant** reads a season they played, by id, on `games > 0 || rounds > 0`. The `games` half exists because the rule previously authorised on `rounds` alone and every ex-member of a timer-only club was pushed a season summary and denied when they opened it (LEDGER `seasonsummary-participant-rounds-only` ; confirmed three times independently). `seasonSummary.test.mjs:131` — "and a timer-only season, where nobody has a single משחקון" — is the regression test, and it passes.
- A departed player may *not* list, may not read a later season their zeroed row still appears in, and may not read a season they never played.
- `seasonCards` is member-only including the list path; nobody writes a card, an archive or a title; nobody deletes one; nothing is public to the internet.
- A player cannot award themselves a title, on their own profile or anybody else's, nor overwrite one they did win.
- A club cannot be **born** holding a `seasons` block (`allow create at firestore.rules:324 ends ! ('seasons' in request.resource.data)`), which closed the door on a client planting `enabled: true` and having the hourly rollover adopt it.

What it leaves free:

- `seasonId` on `/games` is **not pinned anywhere**. `grep -c seasonId firestore.rules` = 0 outside two comments, and no rules test touches it. A member can create a game pre-stamped with any season id and the server's stamp is write-once, so it never corrects it. The resulting orphan (a game stamped `s3` on a club that knows only `s1 / s2`) belongs to no scope and is unrepairable: finished games are client-read-only.
- Only one club shape. The fixtures are one group, one member, one outsider, one departed player.
- The `/groups` update rule is tested only through the budget ceiling described above.

The open questions

These are the decisions that are genuinely unresolved — not defects with a known fix, but forks where more than one answer is defensible and the code has silently taken one. They are ordered by how much of the feature moves when the answer changes.

1. Should the awards denominator be the season's length or the best attendance? Today it is the best attendance: `awardsDenominatorOf ( seasonRollover.ts:133 )` returns `Math.max(...players.games)` , which on the one real season is **19** against a season length of **22**. The gate is `games >= ceil(D/2)` , so 19 gives a threshold of 10 (45% of the season) where the Hebrew copy promises half; and `minPenaltyAttempts(19) = 2` against `(22) = 3` , which is the entire reason מלך הפנדלים went to a player with a 2-of-2 record instead of a 2-of-3 one. *Season length* matches the copy and makes "attended every evening" meaningful, but the numerator (`games` , counted since 22.06) and that denominator (`eveningsSealed` , born 25.08) come from different eras, so on a club whose history predates the counter the gate opens far wider than "half the season" — the code comment at `seasonRollover.ts:114–121` records that as the reason the denominator was changed, though the club it was calibrated against is not named and no reproducible measurement of it survives. *Best attendance* keeps both sides in the same units but is self-referential: `mostLoyal` is the maximum of the same array, so the loyalty winner always clears its own gate and `winner.value / D` is identically 1.0 — which is why `awardsDenominator` was written to the card for half a day and then removed, and why platinum on כתר ההתמדה

is currently unreachable for anyone who missed one night. A third answer exists: keep attendance for *eligibility* and length for every *share* the app displays, which is roughly where the code has landed by accident, but has never been stated as the rule.

2. Should the MVP have a sample-size gate? `mvpAvg` is `eveningScoreSum / eveningScoreCount` (`seasonRollover.ts:516`) with no minimum on the count, ranked by `leaders(..., MVP_SCALE_FLOOR = 6)`. On the real club, `eveningScoreCount` runs 2–3 against 17–19 evenings attended, and all seven members tied at exactly 6.0 — the sentinel the scale returns for "played no mini-games". *No gate* is the status quo and is simple. *The same* `ceil(D/2)` *gate the other titles use* is consistent, but on any club whose evening-score accumulator has been live for less than a season it awards nobody, and the card's current Hebrew blames the players for that. *Shrinking toward the club mean* (an empirical-Bayes prior) ranks honestly but costs explainability: the number that decided the title stops being the number on the player's own card.

3. Should a season's statistics be the live scan or the sealed card? Both exist and they disagree. The card is deterministic, cheap, survives account deletion, and is protected — `allow write: if false` in the rules, `create()` on the server. It is also frozen with whatever was wrong when it was written: seven MVPs at 6.0, and per-player `games` short by exactly their attendance at three evenings that carry no `finishCredited` marker. The live scan is self-correcting — fix the rule and history fixes itself — but cannot answer for a deleted game, changes its answer as evenings age out of the 200-document window, and costs reads on every screen. A third option, *card by default with an admin-visible recompute*, breaks the write-once property both layers currently enforce. Note that the property is already not quite true: `seasonSummary/HhzIwmjM1i5HS0Ght3p__s1` has `createTime 2026-09-17T16:12:08Z` and `updateTime 2026-09-18T11:41:41Z` — it was repaired by hand a day later, which is exactly the operation the design forbids.

4. Should guest pairs be archived? 293 of the 314 `communityPairStats` rows on a seven-player club are guest pairs (93%). The close reads all 314, archives none of the guests, zeroes them, and the undo can only restore what was archived — so a reopen loses them permanently. *Archive them* makes the undo lossless, at the cost of document size (`MAX_ARCHIVED_PAIRS = 1200`, ~250–300 bytes per entry against Firestore's 1MB ceiling) and of freezing a "partner" who is a different human next week, since a guest identity is per-evening. *Delete them at close* removes the 15× read amplification — one close read 314 documents and archived zero — at the cost of the club's lifetime chemistry history. *Never create them* is cheapest of all but the live evening's chemistry card is the feature they exist for. The question is really "is a guest a person or an event", and the data model has not decided.

5. Should eveningsSealed be backfilled? `clubRecords.eveningsSealed` is 10 on a club that has played 22 evenings, because the counter began on 26.08.2026; `seasons.roundsAtStart` is 7, and `10 - 7 = 3` is the number two separate shipped code paths archived for a season everyone had watched reach 22. *Backfill it per club from the games scan* makes the number mean what its name says, but it is the denominator of every running rounds-cadence season: moving it moves every club's finish line at once, and the `playedRounds` mirrors seeded from it would all need re-seeding in the same operation. *Leave it and rename it* ("evenings since 26.08") is honest and cheap but leaves every rounds target measured in a unit the card does not show. *Retire it in favour of stamped games* removes the era problem but reintroduces the one the counter was created to solve: deleting a game would silently walk a season's progress backwards.

6. Should the three counters be collapsed into one derivation? There are three answers to "how many מחזורים": `clubRecords.eveningsSealed` (lifetime counter), `seasons.playedRounds` (per-season mirror), and a scan over stamped games filtered by `didEveningHappen`. `seasonCounterReconciliation.test.ts` exists

precisely because nothing compared them and all three disagreed. *One derivation from the games* is the truthful answer and costs an unbounded scan plus sensitivity to deletion. *Keep the mirror as the display number and make the counter internal* is the smallest change and leaves two numbers that can still drift, held together by a single test over a single club's fixture. *Event-source it* — one immutable document per credited evening in a `seasonEvenings` subcollection — gives idempotency, a repairable history and a cheap count, at the price of a migration and one extra write per seal.

7. Should `מחזור` and `משחקון` be one unit in the data model? They are two today (`completedRounds` vs `totals.rounds` : 22 and 37 on the same season), and the most recent commit at HEAD is the sixth pass at fixing labels that confused them. *Keep both* is correct and requires the copy discipline no test enforces. *Collapse to evenings* makes every timer-only club — the common club — coherent and throws away the advanced mode's per-mini-game record. *Derive the display unit from the club's mode* removes the confusion at the cost of a screen that reads differently for two clubs looking at the same number.

8. What is `totals.rounds` actually counting? Three measurements of the same club's mini-games disagree and nothing in the repository reconciles them: `communityStats.rounds` sealed into s1 is 37, the `committedRounds` latch documents across the surviving games number 35, and the `roundHistory` documents number 31 (§7). Four of the six missing rounds are the club's two oldest evenings, which predate the `roundHistory` subcollection; two belong to games that have since been deleted. The archive prints the 37 as "37 משחקונים" on a write-once card, the chemistry rollup could only ever see the 31, and the 35 is the only one with a per-round document to audit. Until someone decides which of the three is the club's mini-game count, every derived rate — goals per `משחקון`, clean-sheet coverage, the medal steps in §8 — is divided by a number nobody has defended.

9. Should ending a season early be destructive at all? `endSeasonNow` and the sweep take the same irreversible path, and `reopenLastSeason` exists as the compensating transaction — with no time bound, no already-undone guard, and a `seasonWoundBack` stamp that survives it — all 314 pair rows currently carry `seasonWoundBack: 's2'` while s2 is the *running* season, a state that took one deployed guard to render harmless (§5, §6) and that nothing cleans up. The alternative is a non-destructive "sealed" state: compute the archive, leave the live counters alone, and subtract only when the next season's first evening is credited. That makes undo a no-op and removes the resume path entirely, at the cost of every live table reading as cumulative across the seam — which is the behaviour seasons were introduced to end.

Appendix — every defect slug in this document, in one table

Each defect in the sections above carries a unique slug. There are **83** of them (2 P0, 5 P1, 7 P2, 66 without an explicit severity where they were raised). This table is an index, not a summary: search the document for the slug itself to reach the full argument, the code quotation and the production evidence behind it.

The **State** column uses the vocabulary set out under *How to read a defect claim in this document* — and the asymmetry there is the thing to keep in mind while reading it: the backend in production is **HEAD**, the client in the stores is eleven commits behind, so a "fixed and deployed" server half and a "HEAD only" client half can belong to the same defect. A blank means the section did not mark it either way.

#	Sev	§	State	Defect	Where the section opens its case
1	P0	2		live-club-cannot-use-date-cadence	live-club-cannot-use-date-cadence (P0). Until commit ae2c042 (18.09 16:53)
2	P0	10		sealed-season-rank-vs-club-table-nondeterministic	sealed-season-rank-vs-club-table-nondeterministic (P0) is worse than that in the shipped
3	P1	2		enable-sealnow-archives-counter-not-games	enable-sealnow-archives-counter-not-games (P1). completedRounds: played where
4	P1	2		disable-reenable-zeroes-running-season	disable-reenable-zeroes-running-season (P1). It does not. Re-enabling runs the
5	P1	2		season-id-reissued-with-live-stamps	season-id-reissued-with-live-stamps (P1). count advances only on a close, so
6	P1	10		nemesis-victim-tormentor-not-deduplicated	the same person in all four rows he had (nemesis-victim-tormentor-not-deduplicated, P1). The fix
7	P1	10		share-card-claims-a-shared-title-as-its-own	(:209-211). share-card-claims-a-shared-title-as-its-own (P1) is that sharedWith was computed
8	P2	2		rounds-clamp-uses-a-different-history	rounds-clamp-uses-a-different-history (P2). The plan refused a target below
9	P2	2		reenable-inherits-stale-reopenedat	reenable-inherits-stale-reopenedat (P2). The sealNow branch writes

#	Sev	§	State	Defect	Where the section opens its case
10	P2	2		season-plan-refusals-unmapped	season-plan-refusals-unmapped (P2). git show 4de80d0:src/services/seasonService.ts
11	P2	10		assists-per-round-denominator-disagrees-with-club-efficiency-tab	assists-per-round-denominator-disagrees-with-club-efficiency-tab (P2) — the shipped binary divides
12	P2	10		peers-empty-promises-it-will-fill	timer-only club can never keep (peers-empty-promises-it-will-fill, P2).
13	P2	10		archive-completerounds-never-read-by-client	(archive-completerounds-never-read-by-client, P2); the shipped client substitutes totals.rounds,
14	P2	10		share-card-rank-gate-still-prints-last-place	(share-card-rank-gate-still-prints-last-place, P2). One side-effect of the floor is worth naming:
15	P3	10		peer-rows-always-show-the-generic-auto-avatar	(:400-403), not a bug; the bug (peer-rows-always-show-the-generic-auto-avatar, P3) was on the
16	P3	10		share-card-names-one-unit-two-ways	share-card-names-one-unit-two-ways, P3) and to drop the parentheses, because bidi resolves a closing
17	P3	12		cron-is-an-interval-not-a-clock	(P3 cron-is-an-interval-not-a-clock); and the timeout was raised from the 60-second
18	—	1		target-history-never-shown-to-anyone	reader was fixed; nothing renders it even now (ledger target-history-never-shown-to-anyone).
19	—	1		reopen-inherits-the-successor-seasons-target-history	reopen-inherits-the-successor-seasons-target-history: an array element cannot say which season it
20	—	1		undo-close-orphans-the-season-stamps	undo-close-orphans-the-season-stamps, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id.

#	Sev	§	State	Defect	Where the section opens its case
21	—	1		orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id	undo-close-orphans-the-season-stamps, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id.
22	—	1		games-seasonid-unpinned	(ledger games-seasonid-unpinned, emulator-proven with four successful writes).
23	—	1		evenings-22-on-one-screen-0-on-the-next	evenings-22-on-one-screen-0-on-the-next). Note the stamp still reads s1 even though s2 has since
24	—	1		stale-pair-woundback-stamp-skips-next-wipe	half hours before it existed. Ledger: stale-pair-woundback-stamp-skips-next-wipe, ROUND4 §9 — the
25	—	1		club-totals-zeroed-not-subtracted	then violates fifty lines from the bottom. Ledger club-totals-zeroed-not-subtracted. Live:
26	—	1		season1-startsat-is-the-enable-moment	season1-startsat-is-the-enable-moment, season1-range-prints-a-31-hour-lie. Worse, the
27	—	1		season1-range-prints-a-31-hour-lie	season1-startsat-is-the-enable-moment, season1-range-prints-a-31-hour-lie. Worse, the
28	—	1		awards-denominator-derivation-untested	awards-denominator-derivation-untested, repaired-completerounds-silently-contradicts-the-sealed-titles.
29	—	1		repaired-completerounds-silently-contradicts-the-sealed-titles	awards-denominator-derivation-untested, repaired-completerounds-silently-contradicts-the-sealed-titles.
30	—	1		partialdata-only-on-the-sealnow-path	partialdata-only-on-the-sealnow-path.
31	—	1		older-archive-degrades-into-plausible-zeros	older-archive-degrades-into-plausible-zeros: an archive written by an older build yields confident
32	—	1		archive-duo-reader-has-no-floor	archive-duo-reader-has-no-floor, assist-peers-account-for-half-the-season-assists.

#	Sev	§	State	Defect	Where the section opens its case
33	—	1		assist-peers-account-for-half-the-season-assists	archive-duo-reader-has-no-floor, assist-peers-account-for-half-the-season-assists.
34	—	2		approved-rounds-target-is-not-the-one-written	finding approved-rounds-target-is-not-the-one-written from the other side.
35	—	2		reenable-wipes-targethistory	trail of every finish-line move the club ever made — finding reenable-wipes-targethistory.
36	—	3		inseason-tested-as-a-private-copy	inseason-tested-as-a-private-copy).
37	—	3		end-season-now-ignores-playedrounds	card said was 22. (LEDGER end-season-now-ignores-playedrounds; fixed, the fix is
38	—	3		reopen-season1-drops-the-19-unstamped-evenings	reopen-season1-drops-the-19-unstamped-evenings).
39	—	3		mirror-each-ignores-its-own-params	see LEDGER mirror-each-ignores-its-own-params: the describe.each declares five
40	—	3		unstamped-season-1-rule-disagrees-with-the-server-by-19	unstamped-season-1-rule-disagrees-with-the-server-by-19).
41	—	3		seasonid-stamp-orphan-unrepairable	seasonid-stamp-orphan-unrepairable, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id).
42	—	3		group-doc-read-three-times-per-sealed-evening	group-doc-read-three-times-per-sealed-evening). And with the gate added, a game that
43	—	3		no-test-reconciles-the-three-counters	assertion that was missing — LEDGER no-test-reconciles-the-three-counters, which is
44	—	3		nothing-tests-the-sweep	the suite only inside comments (LEDGER nothing-tests-the-sweep).
45	—	4		endson-endsat-one-day-apart	(endson-endsat-one-day-apart). Verified by executing both algorithms with TZ=UTC, as Cloud

#	Sev	§	State	Defect	Where the section opens its case
46	—	4		cadence-with-no-finish-line-never-closes	will ever close. This is cadence-with-no-finish-line-never-closes. The sweep at least now
47	—	4		reopen-extends-a-date-season-by-a-full-length	(reopen-extends-a-date-season-by-a-full-length). rebaseCadence is the function for opening
48	—	4		target-history-entry-carries-no-season-identity	season's history to another (target-history-entry-carries-no-season-identity). Not the
49	—	4		target-history-erased-at-close	target-history-erased-at-close. performSeasonClose (:15911) and endSeasonNow
50	—	4		target-change-can-close-a-season-with-no-confirmation	target-change-can-close-a-season-with-no-confirmation is now <i>half</i> fixed, and the fix
51	—	4		date-retarget-restarts-the-clock	date-retarget-restarts-the-clock. The date branch writes startsOn: todayIn(now) and
52	—	4		date-season-cannot-be-renewed-at-the-same-length	date-season-cannot-be-renewed-at-the-same-length. The save button is gated on
53	—	4		settings-screen-validates-a-live-target-against-zero	settings-screen-validates-a-live-target-against-zero is fixed at HEAD. Per ROUND4 finding 1,
54	—	6		reopen-never-restores-guest-pair-chemistry	Guest pair chemistry (reopen-never-restores-guest-pair-chemistry). A guest id is minted
55	—	6		chemistry-since-not-restored	chemistrySince (chemistry-since-not-restored). The close sets it to the close instant
56	—	6		undo-leaves-dangling-season-summary-pushes	The notification documents (undo-leaves-dangling-season-summary-pushes). createNotificationOnce
57	—	6		pair-wipe-skipped-on-next-close-after-reopen	That is pair-wipe-skipped-on-next-close-after-reopen, and the state it feeds on is in

#	Sev	§	State	Defect	Where the section opens its case
58	—	6		reopen- playedrounds- ignores- unstamped-games	(reopen-playedrounds-ignores-unstamped-games). 4be7f1c filtered on seasonId == lastId
59	—	6		undo-chains- backwards-with- no-limit	undo-chains-backwards-with-no-limit. There is no time bound and no already-undone guard.
60	—	6		club-seasons- block-written- only-after-the- restore-succeeds	club-seasons-block-written-only-after-the-restore-succeeds. The order is: run
61	—	6		reclose-after- undo-notifies- nobody	reclose-after-undo-notifies-nobody. announceSeasonClosed routes through
62	—	7		rank- denominator-of- excludes- attendees	product notes call common; it is the club for which rank-denominator-of-excludes-attendees,
63	—	7		share-card-is- all-zeros-for-a- timer-only-club	share-card-is-all-zeros-for-a-timer-only-club, seasonsummary-participant-rounds-only and
64	—	7		seasonsummary- participant- rounds-only	share-card-is-all-zeros-for-a-timer-only-club, seasonsummary-participant-rounds-only and
65	—	7		season-push- body-promises- three-things-a- timer-club-has- none-of	season-push-body-promises-three-things-a-timer-club-has-none-of were all written. Mock QA cannot
66	—	8		deadlyduo- together-is- collected-and- never-read	deadlyduo-together-is-collected-and-never-read. SeasonPairLine.together (line 61, "Rounds the two were on the same side") is populated at seasonRollover.ts:536 from p.sameTeam and is never r
67	—	8		eligibility- denominator-is- max-attendance	eligibility-denominator-is-max-attendance. Two numbers describing "how long was this season" live in one archive document (19 and 22) and neither is labelled as the other's alternative; awar
68	—	8		loyalty-crown- is-structurally- platinum	loyalty-crown-is-structurally-platinum (fixed at HEAD, and worth the reader's attention as a worked example of the identity). For about a day, seasonRollover.ts wrote awardsDenominator: seas

#	Sev	§	State	Defect	Where the section opens its case
69	—	8		penalty-titles-crowned-on-two-kicks	penalty-titles-crowned-on-two-kicks. A season crown at a perfect 100% off two kicks is what the archive actually contains, and medalTier('penaltyKing', 1.0, ...) grades it platinum (RATE_STEPS
70	—	8		cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate	The same split exists for clean sheets. Defect — cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate. The season title is leaders(eligible, p => p.cleanSheets) — a raw count. Th
71	—	8		mvp-title-held-by-the-entire-club	mvp-title-held-by-the-entire-club. With the original floor = 0, the sentinel cleared the floor and every eligible player tied at exactly 6.0. Production s1 is exactly that:
72	—	8		mvp-medal-can-never-leave-bronze	MVP_STEPS carries an honest comment (lines 72–86) admitting the original 6.5, 7.5, 8.5 was written against an assumed 1–10 scale that the clamp makes impossible, and that a "correction" to 7
73	—	9		limit-200-window-is-a-lifetime-label	limit-200-window-is-a-lifetime-label. The doc-comment at :934-947 is honest that this is a
74	—	9		compare-card-divides-season-goals-by-lifetime-nights	playerCompareService — compare-card-divides-season-goals-by-lifetime-nights is fixed at
75	—	9		closed-season-tooltip-promises-lifetime	closed-season-tooltip-promises-lifetime is true of HEAD and false of every shipped binary. At
76	—	9		appearances-rollup-undercounts-the-scan	appearances-rollup-undercounts-the-scan. mostLoyal deliberately reads the scan for the running
77	—	9		archived-goals-silently-zero	archived-goals-silently-zero. Lifetime club goals are assembled on the client as
78	—	—		70cde240-0d57-4070-a311-9b1df55a1598	70cde240-0d57-4070-a311-9b1df55a1598, updateTime 2026-09-18T18:14:42Z. Its source is 114,528
79	—	—		mvp-title-crowns-the-entire-club	LEDGER / ROUND4 the two audit records this document folds in: round 3 (~130 findings, referenced by slug, e.g. mvp-title-crowns-the-entire-club) and round 4 (30 findings, referenced by n
80	—	10		did-not-play-told-the-season-had-no-games	The hasData split (screen :463-512) is the fix for did-not-play-told-the-season-had-no-games
81	—	12		type-not-implemented	skipped type-not-implemented

#	Sev	§	State	Defect	Where the section opens its case
82	—	13		rules-season-summary	--test-concurrency=1 is load-bearing. Five of the eight suites (firestore, antiHijack, clubIsolation, oldClientCompat, roundHistoryAccess) share projectId: 'demo-soccer' and each calls clear
83	—	13		perform-close-writes-next-season-even-when-nothing-was-archived	Every sixty minutes, unattended, that function may zero a club's communityPlayerStats, communityStats and communityPairStats, write a seasonSummary with create() (which is one-shot), write n